decisionengine

Release 1.7.0rc0.post72+g5eace9d

Fermilab

May 21, 2021

CONTENTS

1 Release Notes 3

2 Developer Documentation 15

3 Jenkins CI pipeline 17

4 Source code 23

5 Indices and tables 67

Python Module Index 69

Index 71

The Decision Engine is a critical component of the HEP Cloud Facility. It provides the functionality of resource scheduling for disparate resource providers, including those which may have a cost or a restricted allocation of cycles

CHAPTER

ONE

RELEASE NOTES

1.1 Release 1.6.1

Patch level (bug fix) release.

1.1.1 Issues fixed in this release

- 306 : /etc/decisionengine/decision_engine.conf as shipped in RPM is wrong format (de0aef3)
- 275 : Running de-client –stop-channel <channel> results in KeyError (59fb44e)

1.1.2 Full list of commits since version 1.6.0

d7ccd8a : doc: fix typo

ac48e50 : updated release notes

de0aef3 : Fix name to match expectations (#305)

59fb44e : postgresql: do not print stack trace for low level library (#309) (#310)

2162bbe : Setuptools uses entry return value as an error msg (#308)

b0fd9fb : 1.6.0 package backports (#302)

1.2 Release 1.6.0

In this release:

- The logic engine has been rewritten in pure python. This removes the last C++ dependency the decision engine had. The build system has been updated accordingly.
- Migrated to setuptools package development library. This build system is the standard vanilla python build system provided with the python distribution. Build configurations have been updated and rpm packaging remains the primary distribution method.
- Completed logging implementation.
- Improvements in error handling and code coverage.
- Improvements in Jenkins and GitHub actions CI/CD pipelines.

1.2.1 Issues fixed in this release

- 44 : Logic Engine doesn't handle missing values gracefully (743effc)
- 253 : Decision engine can sometimes start up at boot time before network name resolution is working (ae04db5)

1.2.2 Full list of commits since version 1.5.0

2551e07 : More coverage for de-client (#296)

dde3945 : Make sure actions either complete in time or die (#295)

381861c : Update Jenkins pipeline configuration (#292)

eb771f4 : Try to cleanup Dockerfile PATH issue (#291)

780cb56 : fix unittest doc

8680942 : update unittest documentation

8154b24 : Fixup sphinx doc (#290)

5f7e13a : enhancements in logging and error handling in dataspace dir (#283)

3d92725 : Add missing runtime requirement (#286)

743effc : Allow conversion from errors to false values in logic-engine expressions. (#284)

124dcab : Inherit version from setuptools_scm if possible (#287)

3669803 : added missing "\" as line continuation

761f1d9 : Drop invalid **init**.py

dc0e71b : migrate to setuptools (#264)

3b6f1bf : Make reaper reset state when starting from stopped proc (#280)

b2f9061 : added ISO-8601 format to time in logging. changed name of function for better clarity. (#279)

0a74fe1 : Improved DE client usage (#281)

ebf53e3 : Added shutdown method to Publisher class (#278)

f95ab6d : Address some flake8/black reports (#274)

1c383b7 : Automatically pull in our settings from about.py (#273)

e71f186 : logging and error handling enhancements to taskmanager directory (#277)

7de9ab9 : Increase Reaper log verbosity (#267)

019d245 : Update actions to follow new best practices (#272)

b84e847 : Avoid possible sync issues in reaper startup (#271)

891975f : Remove vestigial C++ files. (#270)

42e5e1f : enhancements in logging and exception handling in newly added logicengine files (#265)

38effe6 : Ensure the scheduler has started the thread before returning (#269)

db54fa1 : Start testing on PyPy with psycopg2cffi (#223)

cc44058 : Squashed commit of the following: (#263)

d6548e9 : Enhanced logging in the logicengine directory files (#261)

c341bf7 : Better match our workflow with codecov (#260)

1fbe44d : Use 'new' syntax for forward compat (#259)

2294b0b : Do a limited pin on version requirements (#256)

bcda470 : Python implementation of logic engine (#246)

c6721b4 : address comment on RB

ae04db5 : Add Wants and After (network-online.target) dependency

1a96b14 : Fix action repodata

a70cee8 : Move to CodeCov.io

7b16b4e : Add Wants and Requires dependencies (#258)

76c3670 : Move to CodeCov.io (#254)

e7ba013 : Fix action repodata (#255)

d7e72f2 : revert 3.9 test

b04154b : added 1.5.0 release notes

a03da29 : remove 3.9 to see if documentatoin gets generated

1.3 Release 1.5.0

In this release:

- Introduce data product query interface
- Cleanup of Ligic Engine code
- Improvements in error handling
- Improvements in testing and CI

1.3.1 Issues fixed in this release

- 217 , 218 : Add option to de-client –print-product to only print the column names in a data block and-or to print one or more records in key/value format (fe7abcf)
- 240 : Logic Engine call leads to immediate taskmanager segfault exit (d855aa0)
- 239 : implement data product browsing interface (fe9faa9)

1.3.2 Full list of commits since version 1.4.1

d66c54b : Add PEP-0396 metadata (#243)

bfc91a6 : More compat between psycopg2/psycopg2cffi (#248)

f5d31a6 : Cleanup Fixture FIXME (#249)

0dfaf3c : Adding docker documentation (#251)

4b166a2 : Since we are python3 only now, drop python-six compat layer (#252)

fe7abcf : Add format support to de-client (#217) (#241)

df5a3d7 : Add wheel support for easier testing (#247)

7de970d : Add place to inject env if need be (#242)

84e2930 : Fix race in test case (#250)

d855aa0 : Fix fact-lookup to support duplicate names in separate rules. (#245)

51370fb : Resolve fixture 'quickstart' issue (#238)

3ea9129 : Move from TravisCI to raw actions (#235)

fe9faa9 : implement data product browsing interface (#239)

cf0f3c0 : Add support to use custom base docker container to run tests (#234)

d91722f : Compat with psycopg2cffi (#233)

7d15a8c : Test failing source proxy. (#232)

b9a4bbb : Add debug logs for which threads are created #176 (#231)

6e6f4c9 : Updated Jenkins configuration documentation (#229)

2d9fd7b : Log if config passed validation #117 (#230)

60c46d3 : Self-test needs a real namespace to 'import numpy' in new python eval (#228)

a120077 : Test that the doc actually builds during CI (#227)

4b6240a : Extend timeout for coverage combine (#226)

b059696 : Update workflow per changes at github (#225)

7a71cac : Use newer compilers/runtimes (#224)

15ffd93 : Add header for strict includes (#222)

71b141a : Add special PyPy only requirement (#221)

9dbb932 : Move Python C extension to versioned .so file (#220)

ea7ade5 : Migrate from boost-python to pybind11 (#215)

e6b2eae : Add python 3.9 to testing matrix (#219)

04c8f9c : Add the option to print columns types on de-client (#216)

8815dc6 : Logic-engine cleanups (#211)

086d0d5 : fix missing back tick

54cc084 : modified release notes

24744cf : Synchronize access to the task managers (#214)

87a7fda : replde dash with underscore

743d0fd : try sphinx_rtd_theme

18c7909 : added 1.4.0 release notes

ff3d491 : force docker pull when building the docker container to make sure to use an updated base layer (#210)

1.4 Release 1.4.1

In this release:

- Bug fixes to 1.4.0 release

1.4.1 Issues fixed in this release

- 213 : de-client hangs under certain circumstances in version 1.4 and greater (race condition) (84ecfe2)

1.4.2 Full list of commits since version 1.4.0

9799b9a : update release version to 1.4.1

84ecfe2 : Synchronize access to the task managers (#214)

751b6b8 : Address data races; remove need to sleep in unit tests (#205)

1.5 Release 1.4.0

In this release:

- Improvements in error handling and client/server interactions
- Added log rotation by time
- Improvements in code coverage

1.5.1 Issues fixed in this release

- 153 : Have de-client –print-product return different error message if product does not exist (18a950c)
- 171 : yum update on decision engine rpm from python2 to python3 doesn't undo the symlinks (eb85c97)
- 188 : Channel debug info now leaks into startup.log (99d20a5)
- 208 : Error when trying to run reaper in version 1.4.0 (84eccf3)

1.5.2 Full list of commits since version 1.3

84eccf3 : Fix typo in reaper script. (#209)

d836abf : next RC

926944a : Fix coveralls reporting (#198)

b95c323 : Updating base Dockerfile (#199)

d302e31 : Help jsonnet, which doesn't understand PosixPath objects. (#204)

2d791a7 : Test configuration policies. (#197)

236e27a : Ensure items are returned in a stable order (#202)

e974f5f : add pylinit and pycodestyle (#203)

fbe7616 : Test task manager (#196)

686ca80 : require more recent version of pytest-postgresql (#195)

99d20a5 : Fix double-logging problem. (#192)

4ce3d17 : A set of fixtures to simplify unit tests (#183)

65f8052 : Fix typo (#190)

f3a4be8 : Protect against None workers (#187)

ec310fb : remove py3 from package name

7006489 : bump version to 1.4.0rc

158d835 : decisionengine/framework/modules: Fix SourceProxy retries (#184)

1356bf1 : Add support to test any branch in Jenkins (#182)

692fa8e : Add timeout support for unit test on Jenkins (#181)

e3d6e6a : Updated Jenkins documentation to take into account unit tests timeout parametr (#180)

2586a3e : Configuration redesign (#168)

fac984d : Fix error with DBUtils import. Looks like names of modules changed (#175)

7d661ee : Move postgres-specific implementation to postgres source. (#174)

eb85c97 : Rpm (#173)

10fe843 : Adding log rotation by time (#170)

a8d239b : Various improvements. (#167)

d9b92ee : Ignore vim's *.swp files (#166)

d9f72ef : Fix call to shutdown_timeout (and add sample entry to config) (#165)

3161795 : Add drops for items using tables being dropped (#164)

77d186d : Show output of test runtimes in travis (#163)

81820a4 : Allow server to start with no channels. (#161)

49879a6 : DE server and client usability improvements (#160)

de91c4f : Add tests to default and override config (#158)

14df1f6 : Use python fallthrough for options (#159)

ac64a92 : Drop python 2.7 integration tests since we are python3 only (#157)

d963301 : Update Jenkins pipeline to properly test closing PR (#156)

64248cb : Merge 'runtime' tests into running channel tests (#150)

065ad77 : Adding Jenkins pipeline documentation (#155)

18a950c : fix print-product to report non-existing product as such (#154)

6493735 : Fix invalid attribute name (#152)

d953c6a : Remove unnecessary set_start_method call (#149)

c8c9b65 : guarantee that process is killed so test never hang (#147)

f1542b6 : Channel test (#146)

7f349a8 : Fix faulty TaskManager state type (#145)

d50f1c4 : fix logging regression introduced in f5e299969e0611e3480e9fa2782052df… (#142)

becfa26 : Pass the correct type. (#144)

1a60daf : DecisionEngine: fix typo (#143)

9e7b867 : Updating Jenkins pipeline configuration (#140)

e3a6703 : fix regression introduced in f5e299969e0611e3480e9fa2782052df86d7c4ed (#141)

4900bc6 : Restore runtime test. (#139)

0823f3d : Consolidate DE server/client tests into one file. (#138)

4f84435 : A few more access fixes.

160cfd1 : Fix task manager state access.

c00d819 : A few more cleanups.

ec087e2 : Various cleanups

a309ffe : Improvements to DE client CLI.

1.6 Release 1.3.0

In this release:

- Introduced Jsonnet based configuration system
- Improved logging
- Improved coverage of datasource

1.6.1 Full list of commits since version 1.2

239e82c : postgresql: improve SQL query (#133)

668eb1f : Update to make the code compatible with both python and JSON based config files (#129)

afd8837 : Configuration-manager fixes (#128)

571e2be : Remove pip installed system python packages

407d9ed : Update Dockerfile

1fefc69 : Implement unit tests for datablock.py (#122)

43c8d7a : Adjust global configuration to include program-option values. (#126)

2840813 : Switch to Jsonnet configuration system (#125)

5c4ae0e : logging changes: added config file and command line interface (#124)

6697f22 : Further config-manager testing and factorizations. (#123)

fa89fd0 : Insulate multiprocessing test from parent environment. (#120)

139a537 : Allow empty base directory for log file. (#119)

f14d40c : Factorize configuration-loading steps. (#118)

e00afee : Enhance testing and error reporting of ConfigManager (#117)

c3d1be3 : Python 3 upgrades. (#116)

e7399af : Header fix (#114)

0456abf : Adding editor config file, see https://editorconfig.org/ (#115)

82112d1 : Dockerfile: fetch osg 3.5 repo rpm (#113)

97c21b1 : osg version 3.5 (#112)

33f28a8 : Introduce jsonnet dependency (#110)

3f8b55e : improve server error handling (#108)

f15588e : added 1.2.0 release notes

b433325 : Remove unnecessary 'main' functionality. (#107)

1.7 Release 1.2.0

In this release:

- Swithed to python3
- Improved coverage
- Database data retention : added reaper to remove data older than configurable number of days
- Improved logging

1.7.1 decisionengine

3dfe167 : Jenkins pipeline improvements (#106)

22a7073 : pull request for review request 137 (#105)

cafffb2 : Make it possible to run code directly (for tests), and (#100)

802e98b : replace psycog2 witt psycopg2-binary (#101)

573ce8f : Jenkins pipeline improvements (#99)

9d08835 : Run coveralls even under failed state (#97)

bc1df4b : Add tests for PostgreSQL datasource (#71)

c1ac391 : Fix missing py-modules.html (#96)

8dbfdee : Setup gh-pages doc workflow (#94)

cd4a01a : Doc (#93)

673080d : set version to 1.2.0 (for now). Supply conf file that corresponds to (#91)

f912225 : Db (#92)

dc8b68a : Add reaper to the RPC (#83) (#90)

29ade91 : adding .Jenkinsfile with Jenkins pipeline configuration (#86)

c1dfe5c : Don't exclude E1004 from pylint, do exclude line breaks (#89)

440f949 : Fix varname (#88)

313d135 : Compress (#87)

6b8dc4b : Revert "Add reaper to the RPC (#83)"

dbea8e5 : Update utils.sh so pytest will complete.

e848316 : Update to postgresql11

7f4b805 : Add reaper to the RPC (#83)

0ba2c51 : remove astpp module and depedencies it pulls in (#81)

6b8eab9 : don't track test coverage of tests (#80)

0da18ec : made reaper.py executable

aca24a3 : make reaper.py executable, make symbolic link to it from /usr/bin (#72)

0202acf : Implementation of data reaper (#70)

16b6be1 : Simple changes for Python 3 deployment (#69)

fd2418c : Fix warnings caught by PEP-8 Speaks.

d16359b : Python 3 (and other) simplications.

3c7b6b7 : Only run Github Actions for python3.6 (#68)

453cbba : Update README.md

b27ed53 : remove unnecessary (and atually harmful) python shebang (#66)

1.7.2 decisionengine_modules

30d928b : clone version 1.2.0 of decisionengine

ae7c5a6 : Jenkins pipeline improvements (#236)

310befd : T198 (#235)

a65886d : Fix import as reported in : https://github.com/HEPCloud/decisionengin… (#232)

93711cc : Run coveralls even if tests fail (#229)

03d763a : Jenkins pipeline improvements (#230)

f48d30f : Fix/223 (#228)

c8aa262 : github ticket 199 (#222)

0323bda : Address : https://github.com/HEPCloud/decisionengine_modules/issues/224 (#226)

62e4df6 : Add support to run CI on Jenkins (#221)

5ab1541 : bump master version to 1.2.0 (for now) (#219)

bc19c65 : decisionengine_modules/NERSC: Added retry loop for NERSC API Calls (#220)

41a50de : Sync up pep8speaks and run_pylint.sh with decisionengine settings (#218)

db4634f : silence pylint error (#217)

1b95141 : Fix whitespace around operator error

746ea38 : ignore W503

8a8b5f4 : remove unused variable

a6668bf : fix PEP8 warnings

13773ee : address pep8 warnings

6bea4ca : silence pylint error

f589895 : Pass sort=True parameter to fix future warning (#215)

a1d0507 : fixing pep8 warning

a10bd17 : debugging one import error

ec501ad : make coveralls.io links work

deab1a7 : T201 (#204)

69f2645 : Add coveragerc

6d8a5f5 : decisionengine_modules/NERSC: Make Nersc API call backward-compatible with old config (#196)

a7e0af9 : Only run Github Actions for python3.6 (#24)

1.8 Release 1.1.0

In this release:

- Fixed. https://github.com/HEPCloud/decisionengine_modules/issues/108 "Supply Postgres script to delete fields in main database before a certain date"
- significant code cleanup and pep8 compliance
- unit test work
- CI (GitHub actions and Travis) is introduced

commits

f894b1d : Skip unittest (#77)

632e64b : Add ipython

f681a79 : Make python 2.7 tests run on 1.1 branch

d6a32c0 : implementation of data reaper (#75)

2ad8614 : Use sparse checkout for first checkout to get .github/actions (#65)

812f032 : Cat output of pytest log Exit pylint entrypoint with the line count of pep8 and pylint logs Deal with (detach from . . .) Only tar up (S)RPMS dirs for rpm build.

6b05ec7 : Fix errors reported by run_pylint (#62)

d9f5b66 : Setup pep8speaks

c3b8ac2 : Run github actions as non-root uid. Install packages in virtualenv and remove system rpms.

ae01f9e : Support Python 3 for Boost Python

579761c : Support Python 3 for Boost Python

044b979 : Remove unnecessary using declarations.

00f6d00 : Add extra header dependency due to Boost Python ommission.

24e0795 : Apply clang-format

17c17f9 : Remove JSON dependency.

faa0b22 : Massive cleanup.

07b555f : Updates to Github Actions to allow building with python3.6

fef6c11 : Fix errors when running pylint.sh multiple times

da6f077 : Autopep8 -i fixes

39fe5b3 : TaskManager: fix calling log_exception with correct number of arguments and minor format changes to reduce PEP8 warnings

17396da : logicengine: get rid of compuler warnings

01dc3d1 : Only track what we need

b609d73 : Configure coveralls (and some minor cleanup)

bd9ed5e : Many C++ cleanups

2a61876 : Add Badges

c864f27 : Do not call pytest fixtures directly.

307db5f : white space fix

882b58f : fix unit tests

1da687c : Replace Boost facilities with C++ STL ones.

5a6e6b1 : Run tests on push

8404245 : Add missing Boost regex library dependency.

ceb5fe7 : Apply clang-format to files that were missed earlier.

3de9940 : Apply clang-format to C++ code.

8a8f560 : Cache venv directory instead

ad017ce : Build private boost for testing

928c64a : Test pip cache

358939a : Adjust CMakeLists.txt files to use correct Python versions

9f0ddb3 : Add pylint github action.

5e6ce4a : Remove more unused C++ files.

63717fe : Setup travis to use new cmake var

74fab2a : Use cmake argumement -DPYVER=3.6 to build python3 library https://fermicloud140.fnal.gov/reviews/r/31/

843f30c : Minor cleanups per travis-lint

a538cac : Remove unused C++ files.

4c9d125 : Update repo where action is taken from

87fb2d9 : Update rpms installed in docker image. Update entrypoint.sh to use cmake3.

199ee87 : Find python3 libraries using cmake3 from epel rpm Also need to install python3-devel

4c79d2c : Remove unnused GNUmakefiles.

94342ee : Add unit test as a Github Action

1a0e102 : more advanced travis.yml

0be413f : Add helper file for pip

7794327 : Make recursive import happy

7005c78 : Add simple target

de8b0fa : python3 compliance: replace string.join() where appropriate, handle UserDict

2662e6c : note required packages

3b87119 : Add missing header includes.

3e79b84 : Remove defunct code and its tests

b1dbe1a : Ensure attribs are defined at **init**

c4ad78a : Correct logger arguments do avoid duplicate string parse

a8dcc67 : Remove unused imports (per pylint)

d3502b5 : Remove obsolete CVS directories.

d744111 : add six module to the list of required modules

0a9b1e8 : Fix class declaration

b83157e : Handle metaclasses

549f33b : Add config for Travis CI

ee71044 : Drop trailing white space

3f82af6 : Python3 forward compatible syntax

28bf291 : Add safe (for python 2.7) python3 compatible syntax

1d1d76f : prepare for python3

CHAPTER

TWO

DEVELOPER DOCUMENTATION

CHAPTER

THREE

JENKINS CI PIPELINE

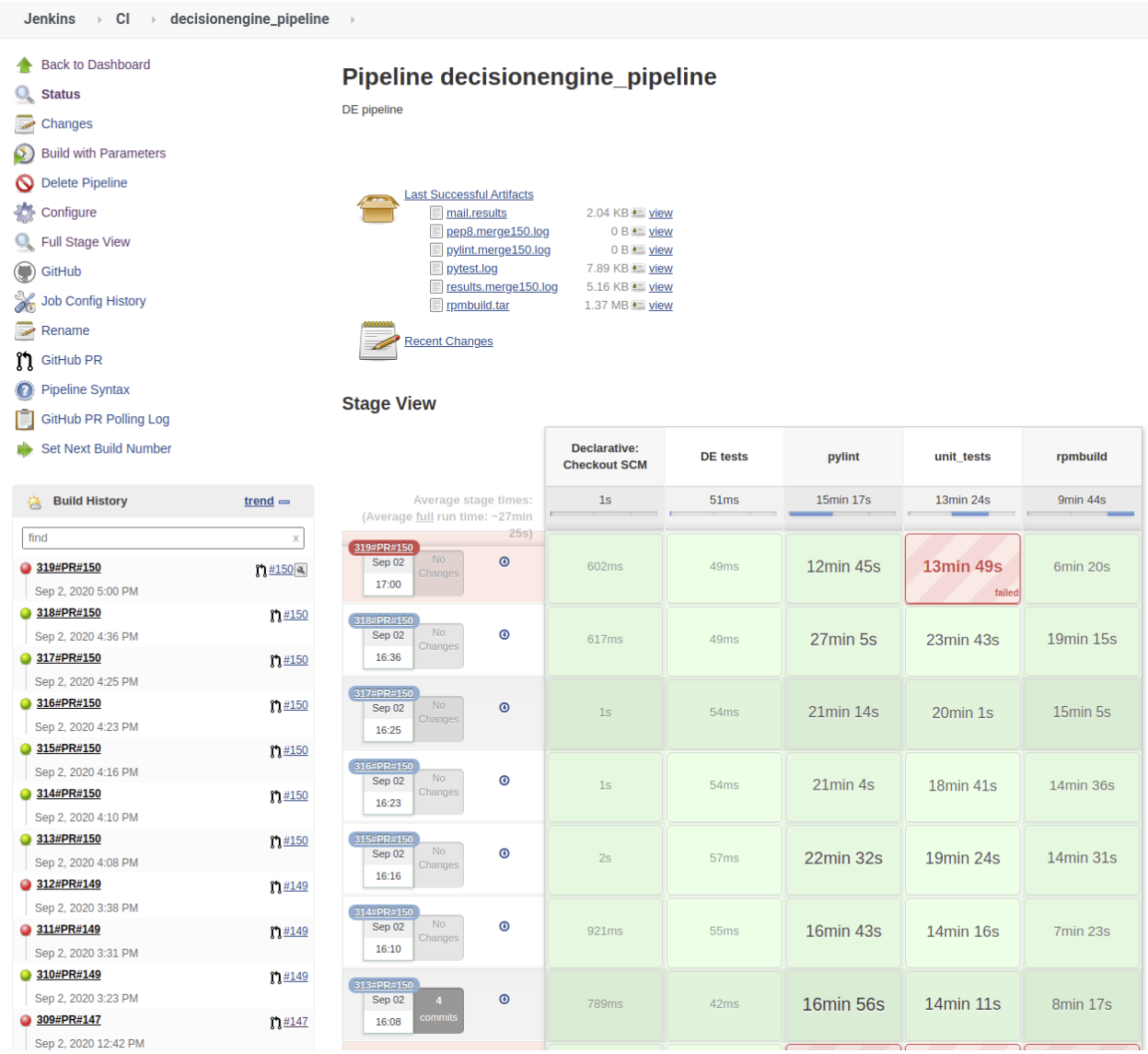
3.1 Decisionengine CI with Jenkins pipeline

Jenkins dashboard with Decisionengine framework CI results is available here.

A CI build is triggered any time a PR is created/closed or a commit is made to an existing PR.
There are also *nightly CI builds* to test a list of predefined branches.

The Jenkins pipeline runs *pylint* and *unit_tests* test suites alongside the *rpmbuild* stage.

The Jenkins dashboard looks like this:



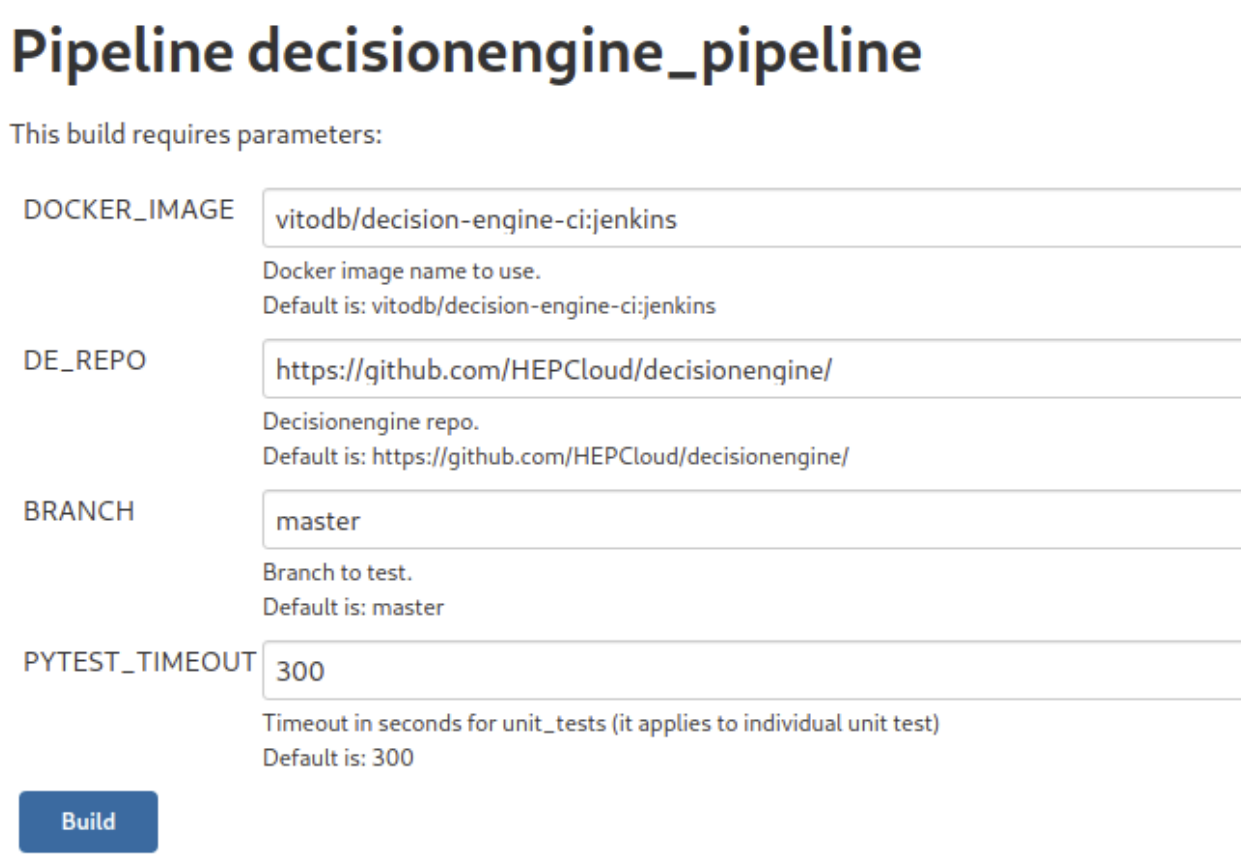
On the bottom left side there is the list of recent CI builds that are named after the PR or the branch tested.
On the bottom right side the dashboard shows for each CI build detailed status for each test suite.

Hovering the mouse over the *status box* for each CI build stage, a tool-tip with a button to access log details shows up.

Next to the build number the symbol gives access to a menu with the list of artifacts stored for that build. Those artifacts include logs and the tarball with RPMs.

From the panel on the left side it is possible to access the PR on GitHub by clicking on the PR icon that looks like this #142 .

On occasion it could be useful to trigger a manual CI build to test a branch on the official DE GitHub repository or on the user fork. For this purpose, on the top left panel the user can click on the Build with Parameters button, and this panel shows up



the user can modify these parameters to customize what code to test with the CI build.

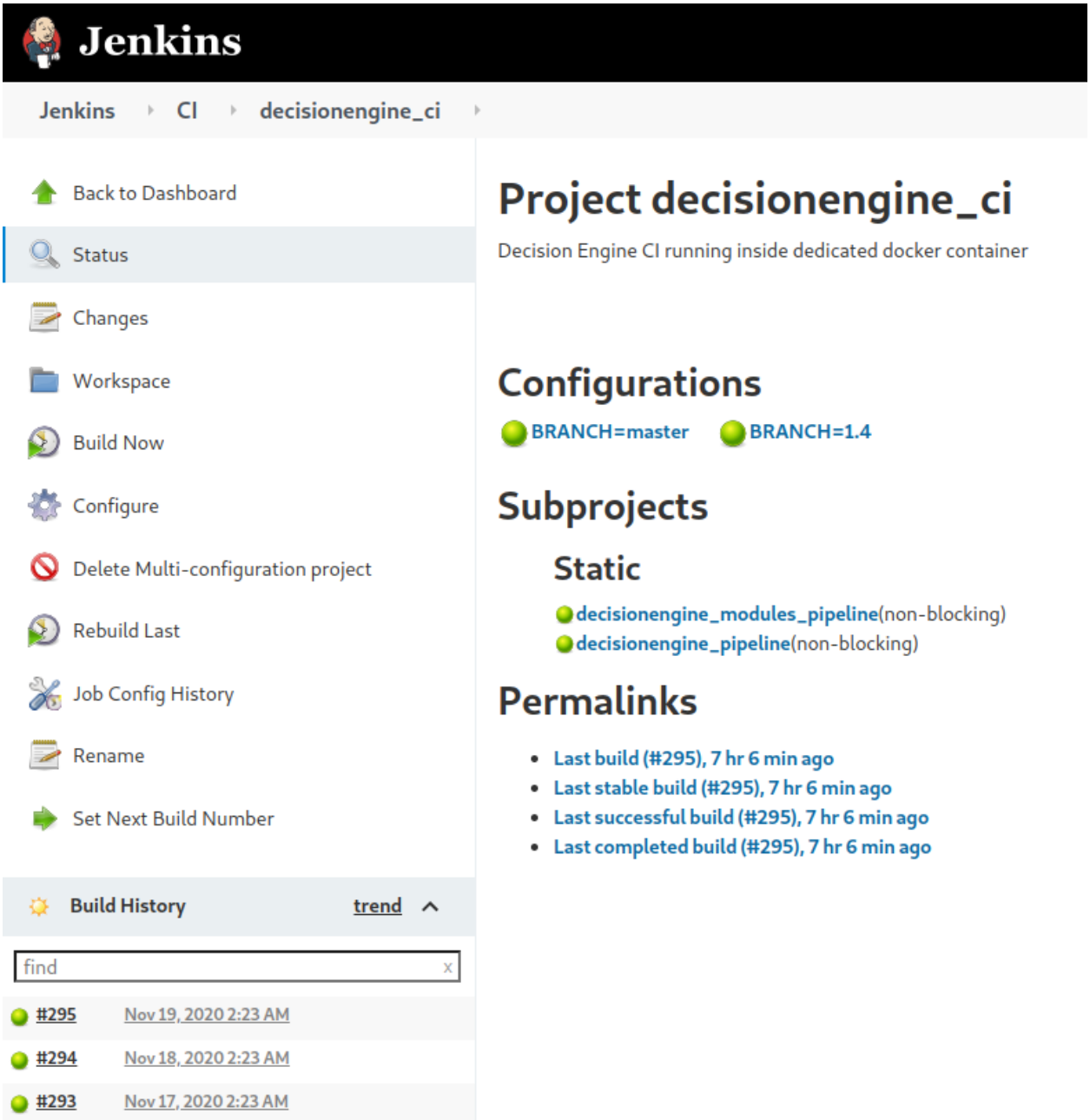
The *DE_REPO* parameter can point to the user fork or to the main repository.
The *BRANCH* parameter can point to the desired branch to test.
The *PYTEST_TIMEOUT* parameter is the timeout in seconds for *unit_tests*.

When ready, by clicking on the *Build* button, the CI build will start.

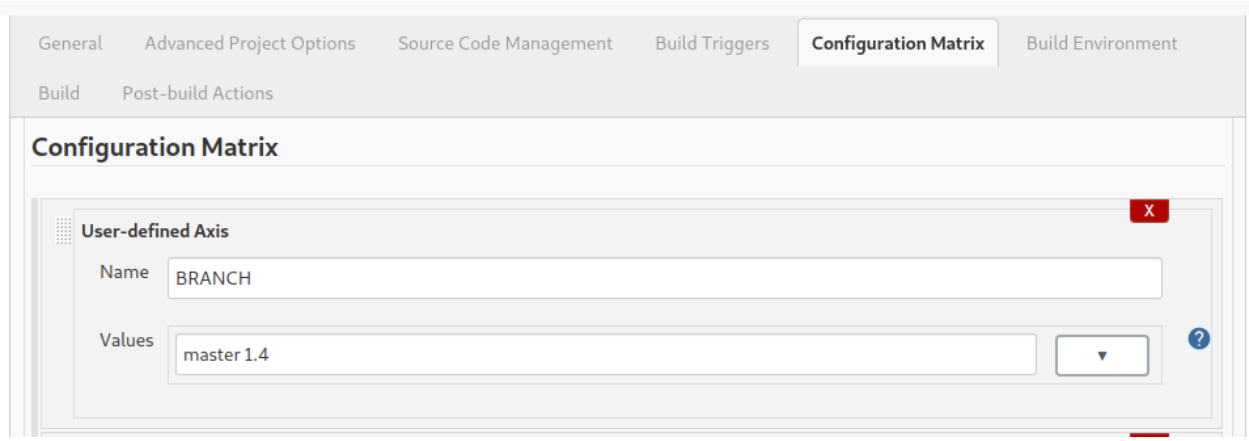
The pipeline configuration is part of the decisionengine repo.

3.1.1 Nightly CI build configuration

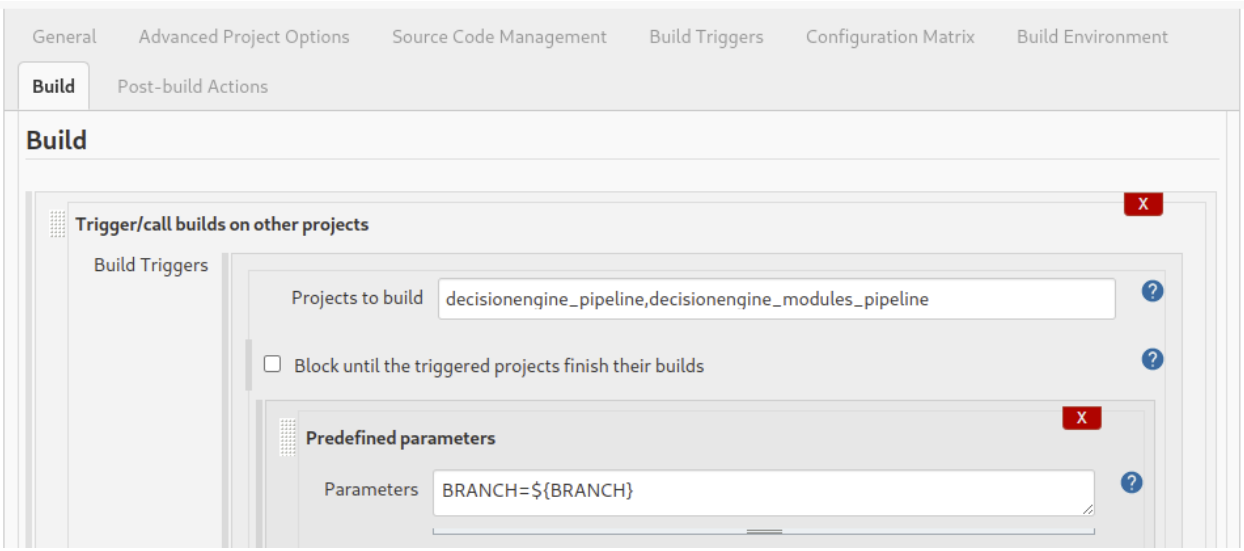
The nightly CI build for Decisionengine framework uses this Jenkins project that triggers a CI build using the Jenkins pipeline described above to test a list of predefined branches.



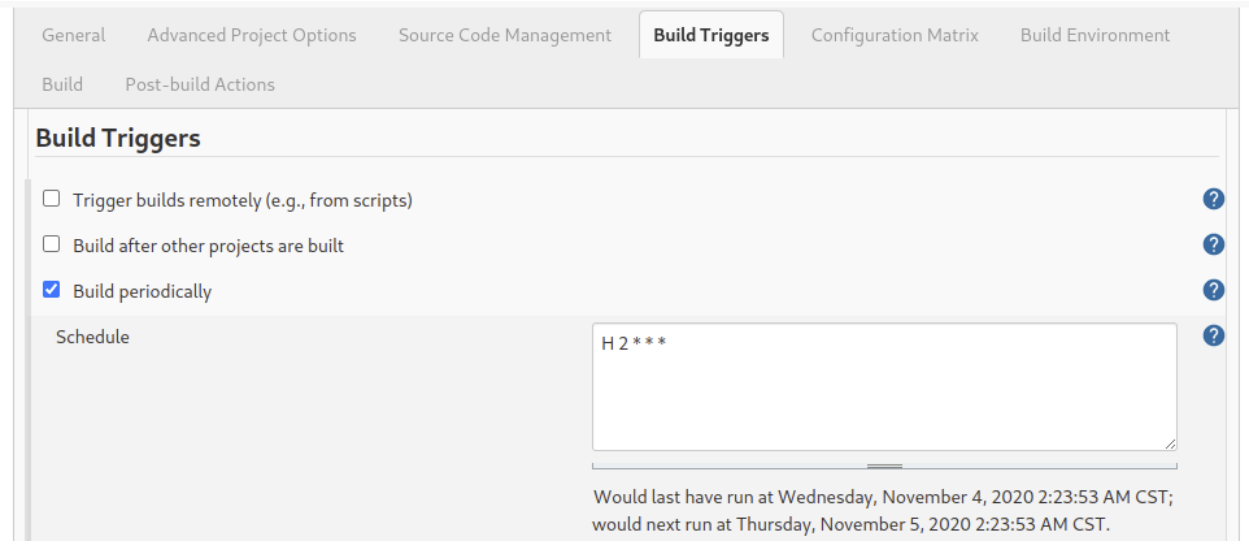
Branches to test are defined using the project matrix as shown in the picture below.
Each branch in the list (here *master* and *1.4*) spawns an independent CI build.



In the *Build* section of the configuration it is set the list of Jenkins subprojects to be triggered, in this case we have *decisionengine_pipeline* and *decisionengine_modules_pipeline*.
The *Parameters* text box is used to override parameters of each Jenkins subproject with a custom value.
In total this Jenkins project triggers 4 CI builds, i.e. 2 branches X 2 Jenkins subprojects.



Finally the *Build Triggers* section is used to setup the schedule for the periodic build,
in this case it is scheduled to run at about 2 AM.
Jenkins will choose the actual time depending on the actual load on the system.


General
Advanced Project Options
Source Code Management
Build Triggers
Configuration Matrix
Build Environment
Build
Post-build Actions
Build Triggers
Trigger builds remotely (e.g., from scripts)
Build after other projects are built
Build periodically
Schedule
H 2 * * *
Would last have run at Wednesday, November 4, 2020 2:23:53 AM CST;
would next run at Thursday, November 5, 2020 2:23:53 AM CST.

CHAPTER

FOUR

SOURCE CODE

4.1 Welcome to decisionengine's documentation!

4.1.1 decisionengine package

Subpackages

decisionengine.framework package

Subpackages

decisionengine.framework.config package

Subpackages

decisionengine.framework.config.tests package

Submodules

decisionengine.framework.config.tests.test_config module

decisionengine.framework.config.tests.test_config.**_channel_config_dir**(*relative_dir*)

decisionengine.framework.config.tests.test_config.**_global_config_file**(*relative_filename*)

decisionengine.framework.config.tests.test_config.**load**()

decisionengine.framework.config.tests.test_config.**test_channel_empty_config**(*load*, *capsys*, *caplog*)

decisionengine.framework.config.tests.test_config.**test_channel_empty_dictionary**(*load*, *caplog*)

decisionengine.framework.config.tests.test_config.**test_channel_invalid_modules_list**(*load*, *caplog*)

decisionengine.framework.config.tests.test_config.**test_channel_invalid_modules_no_keys**(*load*, *caplog*)

decisionengine.framework.config.tests.test_config.**test_channel_invalid_modules_string**(*load*, *caplog*)

decisionengine.framework.config.tests.test_config.**test_channel_loading**(*caplog*)

decisionengine.framework.config.tests.test_config.**test_channel_module_missing_all**(*load*, *caplog*)

decisionengine.framework.config.tests.test_config.**test_channel_module_missing_module**(*load*, *caplog*)

decisionengine.framework.config.tests.test_config.**test_channel_module_missing_parameters**(*load*, *caplog*)

decisionengine.framework.config.tests.test_config.**test_channel_names**(*load*)

decisionengine.framework.config.tests.test_config.**test_channel_no_config_files**(*load*)

decisionengine.framework.config.tests.test_config.**test_channel_no_modules**(*load*)

decisionengine.framework.config.tests.test_config.**test_empty_config**(*load*)

decisionengine.framework.config.tests.test_config.**test_empty_dict**(*load*)

decisionengine.framework.config.tests.test_config.**test_empty_dict_with_leading_comment**(*load*)

decisionengine.framework.config.tests.test_config.**test_minimal_jsonnet_right_extension**(*load*, *capsys*)

decisionengine.framework.config.tests.test_config.**test_minimal_jsonnet_wrong_extension**(*load*, *capsys*)

decisionengine.framework.config.tests.test_config.**test_minimal_python**(*load*, *capsys*)

decisionengine.framework.config.tests.test_config.**test_syntax_error_in_config_names_bad_file**(*load*)

decisionengine.framework.config.tests.test_config.**test_wrong_type**(*load*)

decisionengine.framework.config.tests.test_policies module

decisionengine.framework.config.tests.test_policies.**test_channel_config_dir**(*tmp_path*, *monkeypatch*)

decisionengine.framework.config.tests.test_policies.**test_global_config_dir**(*tmp_path*, *monkeypatch*)

decisionengine.framework.config.tests.test_policies.**test_global_config_file**(*tmp_path*, *monkeypatch*)

decisionengine.framework.config.tests.test_policies.**test_valid_dir**(*tmp_path*)

decisionengine.framework.config.tests.test_validconfig module

decisionengine.framework.config.tests.test_validconfig.**_global_config_file**(*relative_filename*)

decisionengine.framework.config.tests.test_validconfig.**test_empty_config**()

decisionengine.framework.config.tests.test_validconfig.**test_invalid_config**()

decisionengine.framework.config.tests.test_validconfig.**test_no_such_file**()

decisionengine.framework.config.tests.test_validconfig.**test_wrong_type_config**()

Module contents

Submodules

decisionengine.framework.config.ChannelConfigHandler module

Manager of channel configurations.

The ChannelConfigHandler manages only channel configurations and not the global decision-engine configuration. It is responsible for loading channel configuration files and validating that the channels have the correct configuration artifacts and inter-module product dependencies.

class `decisionengine.framework.config.ChannelConfigHandler.`**`ChannelConfigHandler`**(*global_config*, *channel_config_dir*)

Bases: `object`

`_load_channel`(*channel_name*, *path*)

`get_channels`()

`load_all_channels`()

Load all channel configurations inside the stored channel-configuration directory.

Any cached configurations will be dropped prior to reloading.

`load_channel`(*channel_name*)

Load a single configuration for a channel with the supplied name.

The behavior is to read a configuration file whose path is:

<cached channel config. dir>/{channel_name}.jsonnet

where the cached channel-configuration directory was stored whenever the ChannelConfigHandler object was created, and {channel_name} is the value of the supplied method argument.

`print_channel_config`(*channel*)

`decisionengine.framework.config.ChannelConfigHandler.`**`_check_keys`**(*channel_conf_dict*)

check that channel config has mandatory keys :type data: `dict`

`decisionengine.framework.config.ChannelConfigHandler.`**`_make_de_logger`**(*global_config*)

decisionengine.framework.config.ValidConfig module

ValidConfig represents a valid JSON document.

The decision engine requires each of its configuration files to be valid JSON. This is achieved by either supplying a valid Jsonnet or JSON document upfront, or by providing a Python dictionary that can be trivially converted to a JSON document.

Vetting of a file for JSON validity happens upon construction of a 'ValidConfig' object. A fully constructed 'ValidConfig' object thus corresponds to a valid JSON document.

class `decisionengine.framework.config.ValidConfig.`**`ValidConfig`**(*filename*)

Bases: `collections.UserDict`

ValidConfig represents a valid JSON configuration in the form of a dictionary.

In addition to the normal dictionary operations, users may call 'dump()' to print out in a string form the JSON configuration.

`_abc_impl = <_abc._abc_data object>`

dump()
Print dictionary data to a valid JSON string.

`decisionengine.framework.config.ValidConfig.`**_config_from_file**(*config_file*)

`decisionengine.framework.config.ValidConfig.`**_convert_to_json**(*config_file*)
Attempt to convert JSON non-compliant configuration into a compliant one.

This is a temporary facility to aid the migration of Python-based configurations to Jsonnet-based ones. Python dictionaries that are similar in structure to JSON documents are generally trivially convertible.

decisionengine.framework.config.policies module

Decision-engine default configuration policies.

For the decision-engine process, the configuration policies are:

- The global configuration file must be named 'decision_engine.jsonnet' and it must reside in (a) a directory that can be accessed through the 'CONFIG_PATH' environment variable, or (b) the /etc/decisionengine directory.
- All channel configurations must reside in (a) a directory accessible through the 'CHANNEL_CONFIG_PATH' environment variable, or (b) a 'config.d' subdirectory of the /etc/decisionengine directory.

The utilities provided in this module provide simple means of accessing the configuration artifacts according to the policies listed above. Please consult the documentation for each function below for more detailed information.

`decisionengine.framework.config.policies.`**channel_config_dir**(*parent_dir=None*)
Retrieve the channel configuration directory as a pathlib.Path object.

This function returns a path object according to the following precedence rules:

1. If the 'parent_dir' argument is provided, the returned path object will correspond to '{parent_dir}/config.d'.
2. If the 'CHANNEL_CONFIG_PATH' environment variable has been set, the returned path object will correspond to ${CHANNEL_CONFIG_PATH}.
3. If neither 1 or 2 apply, the returned path object corresponds to '{global_config_dir()}/config.d' (see documentation for 'global_config_dir()').

Regardless of the precedence rule used, the returned path object must be a valid directory or an exception will be raised–i.e. if the 'parent_dir' argument is supplied, and the resulting path object is not a valid directory, the function will exit with an exception and not attempt rule 2 or 3.

`decisionengine.framework.config.policies.`**global_config_dir**()
Retrieve global configuration dir as pathlib.Path object.

This is the directory that houses the 'decision_engine.jsonnet' global configuration file.

This function checks that the 'CONFIG_PATH' variable has been set or will use /etc/decisionengine otherwise. If the path exists as a directory, then the directory path is returned as a string; otherwise an exception is raised.

`decisionengine.framework.config.policies.`**global_config_file**(*parent_dir=None*)
Return the pathlib.Path object corresponding to the global configuration.

If supplied, the 'parent_dir' is assumed to be the full path corresponding to a directory containing the 'decision_engine.jsonnet' file. If not provided, the global configuration directory is determined based on the behavior of the 'global_config_dir()' function.

An exception is raised if no 'decision_engine.jsonnet' file is found.

`decisionengine.framework.config.policies.valid_dir(path, scope)`
Throws if the supplied path object is not a directory, otherwise returns the path object.

Module contents

decisionengine.framework.dataspace package

Subpackages

decisionengine.framework.dataspace.datasources package

Subpackages

decisionengine.framework.dataspace.datasources.tests package

Submodules

decisionengine.framework.dataspace.datasources.tests.fixtures module

pytest fixtures/constants

`decisionengine.framework.dataspace.datasources.tests.fixtures.mock_data_block()`
This fixture replaces the standard datablock implementation.

The current DataBlock implementation does not own any data products but forwards them immediately to a backend datasource. The only implemented datasource requires Postgres, which is overkill when needing to test simple data-product communication between modules.

This mock datablock class directly owns the data products, thus avoiding the need for a datasource backend. It is anticipated that a future design of the DataBlock will own the data products, thus making this mock class unnecessary.

decisionengine.framework.dataspace.datasources.tests.test_postgresql module

`decisionengine.framework.dataspace.datasources.tests.test_postgresql.data()`

`decisionengine.framework.dataspace.datasources.tests.test_postgresql.dataproduct()`

`decisionengine.framework.dataspace.datasources.tests.test_postgresql.datasource(postgresql, data)`

`decisionengine.framework.dataspace.datasources.tests.test_postgresql.header(data)`

`decisionengine.framework.dataspace.datasources.tests.test_postgresql.metadata(data)`

`decisionengine.framework.dataspace.datasources.tests.test_postgresql.taskmanager()`

`decisionengine.framework.dataspace.datasources.tests.test_postgresql.test_create_tables(datasource)`

`decisionengine.framework.dataspace.datasources.tests.test_postgresql.test_delete_data_older_than_arg(dat`

`decisionengine.framework.dataspace.datasources.tests.test_postgresql.test_generate_insert_query()`

decisionengine.framework.dataspace.datasources.tests.test_postgresql.**test_get_dataproducts**(*datasource*, *taskmanager*, *data*)

decisionengine.framework.dataspace.datasources.tests.test_postgresql.**test_get_last_generation_id**(*datasou taskman ager*, *data*)

decisionengine.framework.dataspace.datasources.tests.test_postgresql.**test_get_taskmanager**(*datasource*, *taskmanager*, *data*)

decisionengine.framework.dataspace.datasources.tests.test_postgresql.**test_insert**(*datasource*, *dataproduct*, *header*, *metadata*)

decisionengine.framework.dataspace.datasources.tests.test_postgresql.**test_store_taskmanager**(*datasource*, *taskmanager*)

Module contents

Submodules

decisionengine.framework.dataspace.datasources.postgresql module

class decisionengine.framework.dataspace.datasources.postgresql.**Postgresql**(*config_dict*)

Bases: *decisionengine.framework.dataspace.datasource.DataSource*

Implementation of postgresql data source

__query(*query_string*, *values=None*, *cursor_factory=None*)

_abc_impl = <_abc._abc_data object>

_delete(*sql_query*, *values=None*)

_insert(*table_name_or_sql_query*, *record=None*)

_insert_returning_result(*table_name_or_sql_query*, *record=None*)

_remove(*sql_query*, *values=None*)

_select(*query_string*, *values=None*, *cursor_factory=None*)

_select_dictresult(*sql_query*, *values=None*)

_select_getresult(*sql_query*, *values=None*)

_select_tuple(*sql_query*, *values*)

_update(*query_string*, *values=None*)

_update_returning_result(*query_string*, *values=None*)

close()
Close all connections to the database

connect()
Create a pool of database connections

create_tables()
Create database tables

delete_data_older_than(*days*)
Delete data older that days interval :type days: `int` :arg days: remove data older than days interval

duplicate_datablock(*taskmanager_id*, *generation_id*, *new_generation_id*)
For the given taskmanager_id, make a copy of the datablock with given generation_id, set the generation_id for the datablock copy

Parameters

- **taskmanager_id** (`string`) – taskmanager_id for generation to be retrieved
- **generation_id** (`int`) – generation_id of the data
- **new_generation_id** (`int`) – generation_id of the new datablock created

get_connection()

get_datablock(*taskmanager_id*, *generation_id*)
Return the entire datablock from the dataproduct table for the given taskmanager_id, generation_id

Parameters

- **taskmanager_id** (`string`) – taskmanager_id for generation to be retrieved
- **generation_id** (`int`) – generation_id of the data

get_dataproduct(*taskmanager_id*, *generation_id*, *key*)
Return the data from the dataproduct table for the given taskmanager_id, generation_id, key

Parameters

- **taskmanager_id** (`string`) – taskmanager_id for generation to be retrieved
- **generation_id** (`int`) – generation_id of the data
- **key** (`string`) – key for the value

get_dataproducts(*taskmanager_id*, *key=None*)
Return list of all data products associated with with taskmanager_id

Parameters **key** (`string`) – data product key

get_header(*taskmanager_id*, *generation_id*, *key*)
Return the header from the header table for the given taskmanager_id, generation_id, key

Parameters

- **taskmanager_id** (`string`) – taskmanager_id for generation to be retrieved
- **generation_id** (`int`) – generation_id of the data
- **key** (`string`) – key for the value

get_last_generation_id(*taskmanager_name*, *taskmanager_id=None*)
Return last generation id for current task manager or taskmanager w/ task_manager_id.

Parameters

- **name** (`string`) – task manager name

- **taskmanager_id** (string) – task manager id

get_metadata(*taskmanager_id*, *generation_id*, *key*)
Return the metadata from the metadata table for the given taskmanager_id, generation_id, key

Parameters

- **taskmanager_id** (string) – taskmanager_id for generation to be retrieved
- **generation_id** (int) – generation_id of the data
- **key** (string) – key for the value

get_schema(*table=None*)
Given the table name return it's schema

Parameters **table** (string) – Name of the table

get_taskmanager(*taskmanager_name*, *taskmanager_id=None*)
Retrieve TaskManager :type taskmanager_name: `string` :arg taskmanager_name: name of taskmanager to retrieve :type taskmanager_id: `string` :arg taskmanager_id: id of taskmanager to retrieve

get_taskmanagers(*taskmanager_name=None*, *start_time=None*, *end_time=None*)
Retrieve TaskManagers :type taskmanager_name: `string` :arg taskmanager_name: name of taskmanager to retrieve :type taskmanager_id: `string` :arg taskmanager_id: id of taskmanager to retrieve

insert(*taskmanager_id*, *generation_id*, *key*, *value*, *header*, *metadata*)
Insert data into respective tables for the given taskmanager_id, generation_id, key

Parameters

- **taskmanager_id** (string) – taskmanager_id for generation to be retrieved
- **generation_id** (int) – generation_id of the data
- **key** (string) – key for the value
- **value** (object) – Value can be an object or dict
- **header** (Header) – Header for the value
- **header** – Metadata for the value

store_taskmanager(*name*, *taskmanager_id*)
Store TaskManager :type taskmanager_name: `string` :arg taskmanager_name: name of taskmanager to retrieve :type taskmanager_id: `string` :arg taskmanager_id: id of taskmanager to retrieve

tables = {'dataproduct': ['taskmanager_id TEXT', 'generation_id INT', 'key TEXT', 'value BLOB'], 'header': ['taskmanager_id TEXT', 'generation_id INT', 'key TEXT', 'create_time REAL', 'expiration_time REAL', 'scheduled_create_time REAL', 'creator TEXT', 'schema_id INT'], 'metadata': ['taskmanager_id TEXT', 'generation_id INT', 'key TEXT', 'state TEXT', 'generation_time REAL', 'missed_update_count INT'], 'schema': ['schema_id INT', 'schema BLOB']}

update(*taskmanager_id*, *generation_id*, *key*, *value*, *header*, *metadata*)
Update the data in respective tables for the given taskmanager_id, generation_id, key

Parameters

- **taskmanager_id** (string) – taskmanager_id for generation to be retrieved
- **generation_id** (int) – generation_id of the data
- **key** (string) – key for the value
- **value** (object) – Value can be an object or dict

- **header** (Header) – Header for the value
- **header** – Metadata for the value

`decisionengine.framework.dataspace.datasources.postgresql.generate_insert_query(table_name, keys)`

Generate insert query given table name and list of fields

Parameters

- **table_name** (str) – Name of the table to insert into
- **keys** – List of column names

Keys list

Return type str - insert query

Module contents

decisionengine.framework.dataspace.tests package

Submodules

decisionengine.framework.dataspace.tests.test_Reaper module

class `decisionengine.framework.dataspace.tests.test_Reaper.MockSource`

Bases: object

delete_data_older_than(*day*)

class `decisionengine.framework.dataspace.tests.test_Reaper.TestReaper(methodName='runTest')`

Bases: unittest.case.TestCase

logger = <RootLogger root (WARNING)>

setUp()

Hook method for setting up the test fixture before exercising it.

tearDown()

Hook method for deconstructing the test fixture after testing it.

test_fail_bad_config()

test_fail_missing_config()

test_fail_missing_config_key()

test_fail_small_retain()

test_fail_small_run_interval()

test_fail_start_two_reapers()

test_fail_wrong_config_key()

test_just_stop_no_error()

test_loop_of_start_stop_in_clumps()

test_reap_default_state()

test_reaper_can_reap()

test_source_fail_can_be_fixed()

test_start_delay()

test_start_stop()

test_start_stop_stop()

test_state_can_be_active()

test_state_sets_timer_and_uses_it()

decisionengine.framework.dataspace.tests.test_datablock module

class decisionengine.framework.dataspace.tests.test_datablock.**TestDatablock**(*methodName='runTest'*)

Bases: unittest.case.TestCase

pytestmark = [Mark(name='usefixtures', args=('fixtures',), kwargs={})]

setUp()

Hook method for setting up the test fixture before exercising it.

tearDown()

Hook method for deconstructing the test fixture after testing it.

test_DataBlock_constructor()

test_DataBlock_duplicate()

test_DataBlock_get_dataproducts()

test_DataBlock_get_header()

test_DataBlock_get_metadata()

test_DataBlock_get_taskmanager()

test_DataBlock_get_taskmanagers()

test_DataBlock_is_expired()

This test just validates the method/function exists. The stub within our default code should be replaced by a class inheriting from it. That class should have more rational return types.

test_DataBlock_is_expired_with_key()

This test just validates the method/function exists. The stub within our default code should be replaced by a class inheriting from it. That class should have more rational return types.

test_DataBlock_key_management()

test_DataBlock_mark_expired()

test_DataBlock_to_str()

test_Header_constructor()

test_Header_is_valid()

test_Metadata_constructor()

test_Metadata_set_state()

test_compress()

test_zdumps()

test_zloads()

decisionengine.framework.dataspace.tests.test_datablock.**data**()

decisionengine.framework.dataspace.tests.test_datablock.**datasource**(*request*, *postgresql*, *data*)

decisionengine.framework.dataspace.tests.test_datablock.**dblock**(*dspace*, *data*)

decisionengine.framework.dataspace.tests.test_datablock.**dspace**(*datasource*)

decisionengine.framework.dataspace.tests.test_datablock.**fixtures**(*request*, *data*, *dspace*, *dblock*)

decisionengine.framework.dataspace.tests.test_datasource module

decisionengine.framework.dataspace.tests.test_datasource.**test_has_methods_we_expect**()

decisionengine.framework.dataspace.tests.test_dataspace module

decisionengine.framework.dataspace.tests.test_dataspace.**test_dataspace_config_finds_bad**()

Module contents

Submodules

decisionengine.framework.dataspace.datablock module

class decisionengine.framework.dataspace.datablock.**DataBlock**(*dataspace*, *name*, *taskmanager_id=None*, *generation_id=None*, *sequence_id=None*)

Bases: `object`

_insert(*key*, *value*, *header*, *metadata*)

Insert a new product into database with header and metadata

_setitem(*key*, *value*, *header*, *metadata=None*)

put a product in the database with header and metadata

_update(*key*, *value*, *header*, *metadata*)

Update an existing product in the database with header and metadata

duplicate()

Duplicate the datablock and return this new DataBlock. The intent is that at the point the duplication occurs there is only information from the sources in the DataBlock. This also increments the generation_id of this DataBlock.

TODO: Also update the header and the metadata information TODO: Make this threadsafe

Return type *DataBlock*

get(*key*, *default=None*)

Return the value associated with the key in the database

Return type `dict`

get_dataproducts(*key=None*)

get_header(*key*)
Return the Header associated with the key in the database

Return type *Header*

get_metadata(*key*)
Return the metadata associated with the key in the database

Return type *Metadata*

get_taskmanager(*taskmanager_name*, *taskmanager_id=None*)
Retrieve TaskManager :type taskmanager_name: `string` :arg taskmanager_name: name of taskmanager to retrieve :type taskmanager_id: `string` :arg taskmanager_id: id of taskmanager to retrieve :rtype: :obj: *dict*

The dictionary returned looks like : {'datestamp': datetime.datetime(2017, 12, 20, 17, 37, 17, 503210,

tzinfo=psycopg2.tz.FixedOffsetTimezone(offset=-360, name=None)),

'sequence_id': 135L, 'name': 'AWS_Calculations', 'taskmanager_id': '77B16EB5-C79E-45B0-B1B1-37E846692E1D'}

is_expired(*key=None*)
Check if the dataproduct for a given key or any key is expired

keys()

mark_expired(*expiration_time*)
Set the expiration_time for the current generation of the dataproduct and mark it as expired if expiration_time <= current time

put(*key*, *value*, *header*, *metadata=None*)
Put data into the DataBlock

store_taskmanager(*taskmanager_name*, *taskmanager_id*)
Persist TaskManager, returns sequence number :type taskmanager_name: `string` :type taskmanager_id: :obj: *string* :rtype: `int`

class decisionengine.framework.dataspace.datablock.**Header**(*taskmanager_id*, *create_time=None*, *expiration_time=None*, *scheduled_create_time=None*, *creator='module'*, *schema_id=None*)

Bases: `collections.UserDict`

_abc_impl = <_abc._abc_data object>

default_data_lifetime = 1800

is_valid()
Check if the Header has minimum required information

required_keys = {'create_time', 'creator', 'expiration_time', 'scheduled_create_time', 'schema_id', 'taskmanager_id'}

exception decisionengine.framework.dataspace.datablock.**InvalidMetadataError**

Bases: `Exception`

Errors due to invalid Metadata

class decisionengine.framework.dataspace.datablock.**Metadata**(*taskmanager_id*, *state='NEW'*, *generation_id=None*, *generation_time=None*, *missed_update_count=0*)

Bases: `collections.UserDict`

_abc_impl = <_abc._abc_data object>

required_keys = {'generation_id', 'generation_time', 'missed_update_count', 'state', 'taskmanager_id'}

set_state(*state*)

Set the state for the Metadata

valid_states = {'END_CYCLE', 'METADATA_UPDATE', 'NEW', 'START_BACKUP'}

class decisionengine.framework.dataspace.datablock.**ProductRetriever**(*product_name*, *product_type*, *product_source*)

Bases: `object`

decisionengine.framework.dataspace.datablock.**compress**(*obj*)

Compress python object :param obj: python object :return: compressed object

decisionengine.framework.dataspace.datablock.**decompress**(*zbytes*)

Decompress zipped byte stream, convert to string. :param zbytes: byte stream :return: uncompressed string

decisionengine.framework.dataspace.datablock.**zdumps**(*obj*)

Pickle and compress :param obj: a python object :return: compressed string

decisionengine.framework.dataspace.datablock.**zloads**(*zbytes*)

Decompress and unpickle If input is not compressed attempts to just unpickle it

Parameters **zbytes** – compressed bytes

Returns returns python object

decisionengine.framework.dataspace.datasource module

class decisionengine.framework.dataspace.datasource.**DataSource**(*config*)

Bases: `object`

_abc_impl = <_abc._abc_data object>

abstract close()

Close all connections to the database

abstract connect()

Create a pool of database connections

abstract create_tables()

Create database tables

dataproduct_table = 'dataproduct'

Name of the dataproduct table

abstract delete_data_older_than(*days*)

Delete data older that interval :type days: `long` :arg days: remove data older than interval

abstract duplicate_datablock(*taskmanager_id*, *generation_id*, *new_generation_id*)
For the given taskmanager_id, make a copy of the datablock with given generation_id, set the generation_id for the datablock copy

Parameters

- **taskmanager_id** (string) – taskmanager_id for generation to be retrieved
- **generation_id** (int) – generation_id of the data
- **new_generation_id** (int) – generation_id of the new datablock created

abstract get_datablock(*taskmanager_id*, *generation_id*)
Return the entire datablock from the dataproduct table for the given taskmanager_id, generation_id

Parameters

- **taskmanager_id** (string) – taskmanager_id for generation to be retrieved
- **generation_id** (int) – generation_id of the data

abstract get_dataproduct(*taskmanager_id*, *generation_id*, *key*)
Return the data from the dataproduct table for the given taskmanager_id, generation_id, key

Parameters

- **taskmanager_id** (string) – taskmanager_id for generation to be retrieved
- **generation_id** (int) – generation_id of the data
- **key** (string) – key for the value

abstract get_dataproducts(*taskmanager_id*, *key*)
Return list of all data products associated with with taskmanager_id

Parameters **key** (string) – data product key

abstract get_header(*taskmanager_id*, *generation_id*, *key*)
Return the header from the header table for the given taskmanager_id, generation_id, key

Parameters

- **taskmanager_id** (string) – taskmanager_id for generation to be retrieved
- **generation_id** (int) – generation_id of the data
- **key** (string) – key for the value

abstract get_last_generation_id(*name*, *taskmanager_id=None*)
Return last generation id for current task manager or taskmanager w/ task_manager_id.

Parameters

- **name** (string) – task manager name
- **taskmanager_id** (string) – task manager id

abstract get_metadata(*taskmanager_id*, *generation_id*, *key*)
Return the metadata from the metadata table for the given taskmanager_id, generation_id, key

Parameters

- **taskmanager_id** (string) – taskmanager_id for generation to be retrieved
- **generation_id** (int) – generation_id of the data
- **key** (string) – key for the value

abstract get_schema(*table=None*)
Given the table name return it's schema

Parameters **table** (string) – Name of the table

abstract get_taskmanager(*taskmanager_name*, *taskmanager_id*)
Retrieve TaskManager :type taskmanager_name: string :arg taskmanager_name: name of taskmanager to retrieve :type taskmanager_id: string :arg taskmanager_id: id of taskmanager to retrieve

abstract get_taskmanagers(*taskmanager_name=None*, *start_time=None*, *end_time=None*)
Retrieve TaskManagers :type taskmanager_name: string :arg taskmanager_name: name of taskmanager to retrieve :type taskmanager_id: string :arg taskmanager_id: id of taskmanager to retrieve

header_table = 'header'
Name of the header table

abstract insert(*taskmanager_id*, *generation_id*, *key*, *value*, *header*, *metadata*)
Insert data into respective tables for the given taskmanager_id, generation_id, key

Parameters

- **taskmanager_id** (string) – taskmanager_id for generation to be retrieved
- **generation_id** (int) – generation_id of the data
- **key** (string) – key for the value
- **value** (object) – Value can be an object or dict
- **header** (Header) – Header for the value
- **header** – Metadata for the value

metadata_table = 'metadata'
Name of the metadata table

abstract store_taskmanager(*taskmanager_name*, *taskmanager_id*)
Store TaskManager :type taskmanager_name: string :arg taskmanager_name: name of taskmanager to retrieve :type taskmanager_id: string :arg taskmanager_id: id of taskmanager to retrieve

taskmanager_table = 'taskmanager'
Name of the taskmanager table

abstract update(*taskmanager_id*, *generation_id*, *key*, *value*, *header*, *metadata*)
Update the data in respective tables for the given taskmanager_id, generation_id, key

Parameters

- **taskmanager_id** (string) – taskmanager_id for generation to be retrieved
- **generation_id** (int) – generation_id of the data
- **key** (string) – key for the value
- **value** (object) – Value can be an object or dict
- **header** (Header) – Header for the value
- **header** – Metadata for the value

decisionengine.framework.dataspace.dataspace module

class decisionengine.framework.dataspace.dataspace.**DataSourceLoader**(**args*, ***kwargs*)

Bases: object

_ds = None

static create_datasource(*module_name*, *class_name*, *config*)

class decisionengine.framework.dataspace.dataspace.**DataSpace**(*config*)

Bases: object

DataSpace class is collection of datablocks and provides interface to the database used to store the actual data

_tables_created = False

Description of tables and their columns

close()

delete(*taskmanager_id*, *all_generations=False*)

duplicate_datablock(*taskmanager_id*, *generation_id*, *new_generation_id*)

get_dataproduct(*taskmanager_id*, *generation_id*, *key*)

get_dataproducts(*taskmanager_id*, *key=None*)

get_header(*taskmanager_id*, *generation_id*, *key*)

get_last_generation_id(*taskmanager_name*, *taskmanager_id=None*)

get_metadata(*taskmanager_id*, *generation_id*, *key*)

get_taskmanager(*taskmanager_name*, *taskmanager_id=None*)

get_taskmanagers(*taskmanager_name=None*, *start_time=None*, *end_time=None*)

insert(*taskmanager_id*, *generation_id*, *key*, *value*, *header*, *metadata*)

mark_demented(*taskmanager_id*, *keys*, *generation_id=None*)

mark_expired(*taskmanager_id*, *generation_id*, *key*, *expiry_time*)

store_taskmanager(*name*, *id*)

update(*taskmanager_id*, *generation_id*, *key*, *value*, *header*, *metadata*)

exception decisionengine.framework.dataspace.dataspace.**DataSpaceConfigurationError**

Bases: Exception

Errors related to database access

exception decisionengine.framework.dataspace.dataspace.**DataSpaceConnectionError**

Bases: Exception

Errors related to database access

exception decisionengine.framework.dataspace.dataspace.**DataSpaceError**

Bases: Exception

Errors related to database access

exception decisionengine.framework.dataspace.dataspace.**DataSpaceExistsError**

Bases: Exception

Errors related to database access

class decisionengine.framework.dataspace.dataspace.**Singleton**

Bases: type

Singleton pattern using Metaclass http://stackoverflow.com/questions/6760685/creating-a-singleton-in-python

_instances = {}

decisionengine.framework.dataspace.maintain module

class decisionengine.framework.dataspace.maintain.**Reaper**(*config*)

Bases: object

Reaper provides functionality of periodic deletion of data older than retention_interval in days

The class attributes indicate a rational set of defaults that shouldn't be altered by user configuration.

MIN_RETENTION_INTERVAL_DAYS = 7

MIN_SECONDS_BETWEEN_RUNS = 7080

_reaper_loop(*delay*)

The thread actually runs this.

reap()

Actually spawn the query to delete the old records. Lock the state as this task doesn't have a cancel option.

property retention_interval

We have data constraints, so use a property to track

property seconds_between_runs

We have data constraints, so use a property to track

start(*delay=0*)

Start thread with an optional delay to start the thread in X seconds

stop()

Try to stop the reaper, will block if the reaper cannot be interupted.

Module contents

decisionengine.framework.engine package

Subpackages

decisionengine.framework.engine.tests package

Submodules

decisionengine.framework.engine.tests.fixtures module

pytest defaults

`decisionengine.framework.engine.tests.fixtures.DEServer(conf_path=None, conf_override=None, channel_conf_path=None, channel_conf_override=None, host='127.0.0.1', port=None, pg_prog_name='PG_PROG', pg_db_conn_name='DE_DB')`

A DE Server using a private database

`decisionengine.framework.engine.tests.fixtures.DE_DB(request: _pytest.fixtures.FixtureRequest) → psycopg2.extensions.connection`

Fixture factory for PostgreSQL.

Parameters **request** (*FixtureRequest*) – fixture request object

Returns postgresql client

`decisionengine.framework.engine.tests.fixtures.PG_PROG(request: _pytest.fixtures.FixtureRequest, tmpdir_factory: _pytest.tmpdir.TempdirFactory) → pytest_postgresql.executor.PostgreSQLExecutor`

Process fixture for PostgreSQL.

Parameters **request** (*FixtureRequest*) – fixture request object

Return type pytest_dbfixtures.executors.TCPExecutor

Returns tcp executor

decisionengine.framework.engine.tests.test_client_only module

`decisionengine.framework.engine.tests.test_client_only.test_client_err_returned_as_rc()`

no de server is running, so –status should error

`decisionengine.framework.engine.tests.test_client_only.test_client_err_returned_verbose_as_rc()`

no de server is running, so –status should error

`decisionengine.framework.engine.tests.test_client_only.test_client_help(capfd)`

`decisionengine.framework.engine.tests.test_client_only.test_client_with_no_command_says_use_help()`

`decisionengine.framework.engine.tests.test_client_only.test_client_with_no_server()`

`decisionengine.framework.engine.tests.test_client_only.test_client_with_no_server_verbose()`

`decisionengine.framework.engine.tests.test_client_only.test_exclusive_options()`

decisionengine.framework.engine.tests.test_query_tool_only module

`decisionengine.framework.engine.tests.test_query_tool_only.test_query_tool_help()`

`decisionengine.framework.engine.tests.test_query_tool_only.test_query_tool_with_no_server()`

`decisionengine.framework.engine.tests.test_query_tool_only.test_query_tool_with_no_server_verbose()`

decisionengine.framework.engine.tests.test_startup module

`decisionengine.framework.engine.tests.test_startup.`**`_check_override`**(*arguments*)

`decisionengine.framework.engine.tests.test_startup.`**`test_change_port`**()

`decisionengine.framework.engine.tests.test_startup.`**`test_default_config`**()

Module contents

Submodules

decisionengine.framework.engine.DecisionEngine module

Main loop for Decision Engine. The following environment variable points to decision engine configuration file: `DECISION_ENGINE_CONFIG_FILE` if this environment variable is not defined the `DE-Config.py` file from the ``../tests/etc/^ directory will be used.

class `decisionengine.framework.engine.DecisionEngine.`**`DecisionEngine`**(*global_config*, *channel_config_loader*, *server_address*)

Bases: `socketserver.ThreadingMixIn`, `xmlrpc.server.SimpleXMLRPCServer`

`_dataframe_to_column_names`(*df*)

`_dataframe_to_csv`(*df*)

`_dataframe_to_json`(*df*)

`_dataframe_to_table`(*df*)

`_dataframe_to_vertical_tables`(*df*)

`_dispatch`(*method*, *params*)

Dispatches the XML-RPC method.

XML-RPC calls are forwarded to a registered function that matches the called XML-RPC method name. If no such function exists then the call is forwarded to the registered instance, if available.

If the registered instance has a _dispatch method then that method will be called with the name of the XML-RPC method and its parameters as a tuple e.g. instance._dispatch('add',(2,3))

If the registered instance does not have a _dispatch method then the instance will be searched to find a matching method and, if found, will be called.

Methods beginning with an '_' are considered private and will not be called.

`block_until`(*state*)

`block_while`(*state*)

`get_logger`()

`handle_sighup`(*signum*, *frame*)

`reaper_start`(*delay*)

`reaper_status`()

`reaper_stop`()

`rm_channel`(*channel*, *maybe_timeout*)

rpc_block_while(*state_str*)

rpc_get_channel_log_level(*channel*)

rpc_get_log_level()

rpc_kill_channel(*channel*, *timeout=None*)

rpc_print_product(*product*, *columns=None*, *query=None*, *types=False*, *format=None*)

rpc_print_products()

rpc_query_tool(*product*, *format=None*, *start_time=None*)

rpc_reaper_start(*delay=0*)
Start the reaper process after 'delay' seconds. Default 0 seconds delay. :type delay: int

rpc_reaper_status()

rpc_reaper_stop()

rpc_rm_channel(*channel*, *maybe_timeout*)

rpc_set_channel_log_level(*channel*, *log_level*)
Assumes log_level is a string corresponding to the supported logging-module levels.

rpc_show_config(*channel*)
Show the configuration for a channel.

rpc_show_de_config()

rpc_start_channel(*channel_name*)

rpc_start_channels()

rpc_status()

rpc_stop()

rpc_stop_channel(*channel*)

rpc_stop_channels()

start_channel(*channel_name*, *channel_config*)

start_channels()

stop_channels()

stop_worker(*worker*, *timeout*)

class decisionengine.framework.engine.DecisionEngine.**RequestHandler**(*request*, *client_address*, *server*)

Bases: xmlrpc.server.SimpleXMLRPCRequestHandler

rpc_paths = ('/RPC2',)

class decisionengine.framework.engine.DecisionEngine.**StopState**(*value*)

Bases: enum.Enum

An enumeration.

Clean = 2

NotFound = 1

Terminated = 3

`decisionengine.framework.engine.DecisionEngine.`**`_channel_preamble`**(*name*)

`decisionengine.framework.engine.DecisionEngine.`**`_create_de_server`**(*global_config*, *channel_config_loader*)

Create the DE server with the passed global configuration and config manager

`decisionengine.framework.engine.DecisionEngine.`**`_get_de_conf_manager`**(*global_config_dir*, *channel_config_dir*, *options*)

`decisionengine.framework.engine.DecisionEngine.`**`_get_global_config`**(*config_file*, *options*)

`decisionengine.framework.engine.DecisionEngine.`**`_start_de_server`**(*global_config*, *channel_config_loader*)

Create and start the DE server with the passed global configuration and config manager

`decisionengine.framework.engine.DecisionEngine.`**`main`**(*args=None*)

If args is None, sys.argv will be used instead If args is a list, it will be used instead of sys.argv (for unit testing)

`decisionengine.framework.engine.DecisionEngine.`**`parse_program_options`**(*args=None*)

If args is a list, it will be used instead of sys.argv

decisionengine.framework.engine.Workers module

class `decisionengine.framework.engine.Workers.`**`Worker`**(*task_manager*, *logger_config*)

Bases: `multiprocessing.context.Process`

Class that encapsulates a channel's task manager as a separate process.

This class' run function is called whenever the process is started. If the process is abruptly terminated–e.g. the run method is pre-empted by a signal or an os._exit(n) call–the Worker object will still exist even if the operating-system process no longer does.

To determine the exit code of this process, use the Worker.exitcode value, provided by the multiprocessing.Process base class.

`get_consumes`()

`get_produces`()

`get_state_name`()

`run`()

Method to be run in sub-process; can be overridden in sub-class

`wait_until`(*state*)

`wait_while`(*state*)

class `decisionengine.framework.engine.Workers.`**`Workers`**

Bases: `object`

This class manages and provides access to the task-manager workers.

The intention is that the decision engine never directly interacts with the workers but refers to them via a context manager:

with workers.access() as ws: # Access to ws now protected ws['new_channel'] = Worker(...)

In cases where the decision engine's block_while or block_until methods must be called (e.g. during tests), one should used the unguarded access:

with workers.unguarded_access() as ws: # Access to ws is unprotected
ws['new_channel'].wait_until(...)

Calling a blocking method while using the protected context manager (i.e. workers.access()) will likely result in a deadlock.

class Access(*workers*, *lock*)
Bases: `object`

_update_channel_states()

access()

unguarded_access()

decisionengine.framework.engine.de_client module

decisionengine.framework.engine.de_client.**console_scripts_main**(*args_to_parse=None*)
This is the entry point for the setuptools auto generated scripts. Setuptools thinks a return from this function is an error message.

decisionengine.framework.engine.de_client.**create_parser**()

decisionengine.framework.engine.de_client.**execute_command_from_args**(*argsparsed*, *de_socket*)
argsparsed should be from create_parser in this file

decisionengine.framework.engine.de_client.**main**(*args_to_parse=None*)
If you pass a list of args, they will be used instead of sys.argv

decisionengine.framework.engine.de_query_tool module

decisionengine.framework.engine.de_query_tool.**create_parser**()

decisionengine.framework.engine.de_query_tool.**execute_command_from_args**(*argsparsed*, *de_socket*)
Calls the proper function for the arguments passed to de_query_tool.

Parameters

- **argsparsed** (*Namespace*) – Should be from create_parser in this file.
- **de_socket** (*ServerProxy*) – RPC Server Proxy.

Returns Output of the command.

Return type str

decisionengine.framework.engine.de_query_tool.**main**(*args_to_parse=None*)
Main function for de_query_tool

Parameters **args_to_parse** (*list, optional*) – If you pass a list of args, they will be used instead of sys.argv. Defaults to None.

Returns Query result

Return type str

Module contents

decisionengine.framework.logicengine package

Subpackages

decisionengine.framework.logicengine.tests package

Submodules

decisionengine.framework.logicengine.tests.test_cascaded_rules module

`decisionengine.framework.logicengine.tests.test_cascaded_rules.`**`myengine`**`()`

`decisionengine.framework.logicengine.tests.test_cascaded_rules.`**`test_rule_that_does_not_fire`**`(`*myengine*`)`

`decisionengine.framework.logicengine.tests.test_cascaded_rules.`**`test_rule_that_fires`**`(`*myengine*`)`

decisionengine.framework.logicengine.tests.test_construction module

`decisionengine.framework.logicengine.tests.test_construction.`**`test_configuration_with_fact_using_functio`**

`decisionengine.framework.logicengine.tests.test_construction.`**`test_configuration_with_numy_facts`**`()`

`decisionengine.framework.logicengine.tests.test_construction.`**`test_default_construction`**`()`
LogicEngine is not default constructible.

`decisionengine.framework.logicengine.tests.test_construction.`**`test_trivial_configuration`**`()`
Logic engine constructed with trivial rules and facts.

`decisionengine.framework.logicengine.tests.test_construction.`**`test_wrong_configuration`**`()`
LogicEngine construction requires rules and facts; if we don't supply them it is an error.

decisionengine.framework.logicengine.tests.test_duplicate_fact_names module

`decisionengine.framework.logicengine.tests.test_duplicate_fact_names.`**`test_duplicate_fact_names`**`()`

decisionengine.framework.logicengine.tests.test_facts module

`decisionengine.framework.logicengine.tests.test_facts.`**`make_db`**`(`*maximum*`)`

`decisionengine.framework.logicengine.tests.test_facts.`**`test_compound_fact_with_spaces`**`()`

`decisionengine.framework.logicengine.tests.test_facts.`**`test_fact_using_numpy_array`**`()`

`decisionengine.framework.logicengine.tests.test_facts.`**`test_fact_using_numpy_function`**`()`

`decisionengine.framework.logicengine.tests.test_facts.`**`test_fact_with_fail_on_error`**`()`

`decisionengine.framework.logicengine.tests.test_facts.`**`test_fact_with_nested_names`**`()`

`decisionengine.framework.logicengine.tests.test_facts.`**`test_simple_fact`**`()`

`decisionengine.framework.logicengine.tests.test_facts.`**`test_syntax_error`**`(`*caplog*`)`

decisionengine.framework.logicengine.tests.test_fail_on_error module

decisionengine.framework.logicengine.tests.test_fail_on_error.**logic_engine_with_fact**(*fact*)

decisionengine.framework.logicengine.tests.test_fail_on_error.**test_conditional_fact**()

decisionengine.framework.logicengine.tests.test_fail_on_error.**test_fact_with_misspecified_attribute**()

decisionengine.framework.logicengine.tests.test_fail_on_error.**test_fail_on_error**(*caplog*)

decisionengine.framework.logicengine.tests.test_fail_on_error.**test_false_fact_with_spaces**()

decisionengine.framework.logicengine.tests.test_fail_on_error.**test_false_literal_fact**()

decisionengine.framework.logicengine.tests.test_fail_on_error.**test_index_error**()

decisionengine.framework.logicengine.tests.test_fail_on_error.**test_misspecified_fact**()

decisionengine.framework.logicengine.tests.test_fail_on_error.**test_true_fact**()

decisionengine.framework.logicengine.tests.test_fail_on_error.**test_true_literal_fact**()

decisionengine.framework.logicengine.tests.test_pandas_fact module

decisionengine.framework.logicengine.tests.test_pandas_fact.**mydata**(*y*)
Return a 'datablock' surrogate carrying a Pandas DataFrame, and a parameter named 'y' with value y.

decisionengine.framework.logicengine.tests.test_pandas_fact.**myengine**()

decisionengine.framework.logicengine.tests.test_pandas_fact.**test_rule_that_does_not_fire**(*myengine*)
Rules that do not fire do not create entries in the returned actions and newfacts.

decisionengine.framework.logicengine.tests.test_pandas_fact.**test_rule_that_fires**(*myengine*)

decisionengine.framework.logicengine.tests.test_rule_with_negated_fact module

decisionengine.framework.logicengine.tests.test_rule_with_negated_fact.**myengine**()

decisionengine.framework.logicengine.tests.test_rule_with_negated_fact.**test_rule_that_does_not_fire**(*myen*
Rules that do not fire do not create entries in the returned actions and newfacts.

decisionengine.framework.logicengine.tests.test_rule_with_negated_fact.**test_rule_that_fires**(*myengine*)

decisionengine.framework.logicengine.tests.test_simple_configuration module

decisionengine.framework.logicengine.tests.test_simple_configuration.**myengine**()

decisionengine.framework.logicengine.tests.test_simple_configuration.**test_rule_that_does_not_fire**(*myengi*
Rules that do not fire do not create entries in the returned actions and newfacts.

decisionengine.framework.logicengine.tests.test_simple_configuration.**test_rule_that_fires**(*myengine*)

Module contents

Submodules

decisionengine.framework.logicengine.BooleanExpression module

class decisionengine.framework.logicengine.BooleanExpression.**BooleanExpression**(*expr*)

Bases: object

evaluate(*d*)

Return the evaluated Boolen value of this expression in the context of the given data 'd'.

exception decisionengine.framework.logicengine.BooleanExpression.**LogicError**

Bases: TypeError

decisionengine.framework.logicengine.BooleanExpression.**function_name_from_call**(*callnode*)

decisionengine.framework.logicengine.BooleanExpression.**maybe_fail_on_error**(*expr*)

decisionengine.framework.logicengine.FactLookup module

class decisionengine.framework.logicengine.FactLookup.**FactLookup**(*fact_names*, *rules_cfg*)

Bases: object

Establishes a policy for looking up a fact based on the given name.

To wit, the first fact with a given name is the one that is used in the evaluation of all subsequent facts.

As an example, consider the following configuration:

facts: { should_publish: "(True)",

}, rules: {

publish_1: { expression: "should_publish", facts: ["should_publish"]

}, publish_2: {

expression: "should_publish", actions: ["go_to_press"] facts: ["should_publish"]

} retract: {

expression: "not should_publish", facts: ["should_retract"]

}

In the above, the first fact to be evaluated will always be the top-level facts (i.e. those not encapsulated by the 'rules' table). The rules labeled 'publish_1' and 'publish_2' both rely on the 'should_publish' fact in their expressions, and they in turn create their own facts with the same name. FactLookup ensures that 'publish_1' and 'publish_2' will both use the evaluated fact from the top-level 'facts' table.

rule_for(*fact_name*)

Selects rule required to evaluate fact with the supplied name.

Parameters **fact_name** (*str*) – Name of fact for which rule will be selected.

Return type str

Returns Rule name

sorted_rules(*rules_cfg*)

Rules sorted according to rule dependencies.

Parameters **rules_cfg** (*dict*) – rules as specified in logic-engine configuration

Return type list

Returns Rules to be evaluated by the rule engine.

decisionengine.framework.logicengine.LogicEngine module

class decisionengine.framework.logicengine.LogicEngine.**LogicEngine**(*cfg*)

Bases: *decisionengine.framework.modules.Module.Module*

_create_facts_dataframe(*newfacts*)

Convert newfacts dict in format below to dataframe with columns ['rule_name', 'fact_name', fact_value']

facts dict format: 'newfacts': {

'publish_glidein_requests': { 'allow_hpc_new': True, 'allow_foo': True

}, 'dummy_rule': {

'dummy_new_fact': True

}

}

consumes()

Return the names of all the items that must be in the DataBlock for the rules to be evaluated.

evaluate(*db*)

Evaluate our facts and rules, in the context of the given data. db can be any mappable, in particular a DataBlock or dictionary.

Parameters **db** (DataBlock) – Products used to evaluate facts.

evaluate_facts(*db*)

Parameters **db** (DataBlock) – Products used to evaluate facts.

Return type dict

Returns Evaluated fact values (e.g. True or False) for each fact name.

produces()

decisionengine.framework.logicengine.Rule module

class decisionengine.framework.logicengine.Rule.**Rule**(*rule_name*, *rule_cfg*)

Bases: object

In-memory representation of logic-engine rule, relying on parsing utilities in BooleanExpression.

evaluate(*evaluated_facts*)

Evaluates a compiled expression given the supplied facts.

Parameters **evaluated_facts** (*dict*) – Initial fact values (e.g. True or False) for each fact name.

Return type bool

decisionengine.framework.logicengine.RuleEngine module

class decisionengine.framework.logicengine.RuleEngine.**RuleEngine**(*fact_names*, *rules_cfg*)

Bases: object

Engine responsible for evaluating logic-engine rules.

This class is responsible for (a) forming a sorted set of rules that supports dependencies between them, and (b) evaluating the rules according to a specified fact-lookup policy.

execute(*evaluated_facts*)

Evaluates all rules given the supplied facts.

Parameters **evaluated_facts** (*dict*) – Initial fact values (e.g. True or False) for each fact name.

Return type tuple

Returns Actions to be taken based on rule evaluation; new facts produced during that evaluation.

Module contents

decisionengine.framework.modules package

Subpackages

decisionengine.framework.modules.tests package

Submodules

decisionengine.framework.modules.tests.test_LogicEngine module

decisionengine.framework.modules.tests.test_LogicEngine.**test_logicengine_structure**()

The module.Module itself is a bit of a skeleton…

decisionengine.framework.modules.tests.test_Module module

decisionengine.framework.modules.tests.test_Module.**test_module_structure**()

The module.Module itself is a bit of a skeleton…

decisionengine.framework.modules.tests.test_Publisher module

decisionengine.framework.modules.tests.test_Publisher.**test_publisher_structure**()

The module.publisher itself is a bit of a skeleton…

decisionengine.framework.modules.tests.test_Source module

decisionengine.framework.modules.tests.test_Source.**test_source_structure**()
The module.Source itself is a bit of a skeleton…

decisionengine.framework.modules.tests.test_Transform module

decisionengine.framework.modules.tests.test_Transform.**test_transform_structure**()
The module.Transform itself is a bit of a skeleton…

decisionengine.framework.modules.tests.test_de_logger module

class decisionengine.framework.modules.tests.test_de_logger.**TestLogger**(*methodName='runTest'*)
Bases: unittest.case.TestCase

setUp()
Hook method for setting up the test fixture before exercising it.

test_by_nonsense_is_err()

test_by_size()

test_by_time()

decisionengine.framework.modules.tests.test_module_decorators module

decisionengine.framework.modules.tests.test_module_decorators.**test_multiple_consumes_declarations**()

decisionengine.framework.modules.tests.test_module_decorators.**test_multiple_produces_declarations**()

decisionengine.framework.modules.tests.test_module_decorators.**test_supports_config**()

decisionengine.framework.modules.tests.test_module_decorators.**test_wrong_product_names**()

decisionengine.framework.modules.tests.test_module_decorators.**test_wrong_product_types**()

Module contents

Submodules

decisionengine.framework.modules.LogicEngine module

class decisionengine.framework.modules.LogicEngine.**LogicEngine**(*set_of_parameters*)
Bases: *decisionengine.framework.modules.Module.Module*

evaluate(*data_block*)

decisionengine.framework.modules.Module module

class decisionengine.framework.modules.Module.**Module**(*set_of_parameters*)

Bases: object

A skeleton of a module

get_data_block()

get_parameters()

set_data_block(*data_block*)

decisionengine.framework.modules.Module.**consumes**(***kwargs*)

decisionengine.framework.modules.Module.**produces**(***kwargs*)

decisionengine.framework.modules.Module.**verify_products**(*producer*, *data*)

decisionengine.framework.modules.Publisher module

class decisionengine.framework.modules.Publisher.**Parameter**(*name*, *type=None*, *default=None*, *comment=None*)

Bases: object

class decisionengine.framework.modules.Publisher.**Publisher**(*set_of_parameters*)

Bases: *decisionengine.framework.modules.Module.Module*

_consumes = {}

publish(*data_block=None*)

shutdown()

decisionengine.framework.modules.Publisher.**consumes**(***kwargs*)

decisionengine.framework.modules.Publisher.**describe**(*cls*, *program_options=<class 'decisionengine.framework.modules.describe.ModuleProgramOptions'>*)

decisionengine.framework.modules.Publisher.**supports_config**(**args*)

decisionengine.framework.modules.Source module

class decisionengine.framework.modules.Source.**Parameter**(*name*, *type=None*, *default=None*, *comment=None*)

Bases: object

class decisionengine.framework.modules.Source.**Source**(*set_of_parameters*)

Bases: *decisionengine.framework.modules.Module.Module*

_produces = {}

acquire()

post_create(*global_config*)

decisionengine.framework.modules.Source.**describe**(*cls*, *sample_config=None*)

decisionengine.framework.modules.Source.**produces**(***kwargs*)

decisionengine.framework.modules.Source.**supports_config**(**args*)

decisionengine.framework.modules.SourceProxy module

Fill in data from another channel data block

class decisionengine.framework.modules.SourceProxy.**SourceProxy**(*config*)

Bases: *decisionengine.framework.modules.Source.Source*

_get_data(*data_block*, *key*)

_supported_config = {'Dataproducts': (<class 'list'>, None, 'List of data products to retrieve.'), 'channel_name': (<class 'str'>, None, 'Channel from which to retrieve data products.'), 'retries': (<class 'int'>, 10, 'Number of attempts allowed to fetch products.'), 'retry_timeout': (<class 'int'>, 60, 'Number of seconds to wait between retries.')}

acquire()

Overrides Source class method

post_create(*global_config*)

decisionengine.framework.modules.Transform module

class decisionengine.framework.modules.Transform.**Parameter**(*name*, *type=None*, *default=None*, *comment=None*)

Bases: object

class decisionengine.framework.modules.Transform.**Transform**(*set_of_parameters*)

Bases: *decisionengine.framework.modules.Module.Module*

_consumes = {}

_produces = {}

transform()

decisionengine.framework.modules.Transform.**consumes**(***kwargs*)

decisionengine.framework.modules.Transform.**describe**(*cls*, *program_options=<class 'decisionengine.framework.modules.describe.ModuleProgramOptions'>*)

decisionengine.framework.modules.Transform.**produces**(***kwargs*)

decisionengine.framework.modules.Transform.**supports_config**(**args*)

decisionengine.framework.modules.de_logger module

Logger to use in all modules

decisionengine.framework.modules.de_logger.**get_logger**()

get default logger - "decision_engine" :rtype: logging.Logger - rotating file logger

decisionengine.framework.modules.de_logger.**set_logging**(*log_level*, *file_rotate_by*, *rotation_time_unit='D'*, *rotation_interval=1*, *max_backup_count=6*, *max_file_size=200000000*, *log_file_name='/tmp/decision_engine_logs/decision_engine_lo*

Parameters

- **log_level** (str) – log level
- **file_rotate_by** – files rotation by size or by time
- **rotation_time_unit** (str) – unit of time for file rotation
- **rotation_interval** (int) – time in rotation_time_units between file rotations
- **log_file_name** (str) – log file name
- **max_file_size** (int) – maximal size of log file. If reached save and start new log.
- **max_backup_count** (int) – start rotaion after this number is reached

Return type logging.Logger - rotating file logger

decisionengine.framework.modules.de_logger.**set_stream_logging**(*logger_name='decision_engine'*)

This is for debugging. Set stream logging for logger.

Parameters **logger_name** (str) – logger name

Return type logging.Logger

decisionengine.framework.modules.describe module

class decisionengine.framework.modules.describe.**ModuleProgramOptions**(*module_spec*, *cls*)

Bases: object

process_args()

class decisionengine.framework.modules.describe.**Parameter**(*name*, *type=None*, *default=None*, *comment=None*)

Bases: object

decisionengine.framework.modules.describe.**_par_default**(*par_type*, *default_value*)

decisionengine.framework.modules.describe.**_par_type**(*par_type*, *default_value*)

decisionengine.framework.modules.describe.**main_wrapper**(*cls*, *program_options=<class 'decisionengine.framework.modules.describe.ModuleProgramOptions'>*

decisionengine.framework.modules.describe.**supports_config**(**args*)

decisionengine.framework.modules.print_description module

decisionengine.framework.modules.print_description.**_print_comment**(*comment*)

decisionengine.framework.modules.print_description.**_print_type**(*type_or_value*)

decisionengine.framework.modules.print_description.**_print_value**(*v*)

decisionengine.framework.modules.print_description.**_spec_from_file_name**(*filename*)

decisionengine.framework.modules.print_description.**print_consumes**(*cls*)

decisionengine.framework.modules.print_description.**print_produces**(*cls*)

decisionengine.framework.modules.print_description.**print_supported_config**(*module_spec*, *cls*)

decisionengine.framework.modules.print_description.**spec_if_main**(*cls*)

Module contents

decisionengine.framework.taskmanager package

Submodules

decisionengine.framework.taskmanager.ProcessingState module

The ProcessingState class can represent any of the following task-manager states:

BOOT IDLE ACTIVE STEADY OFFLINE SHUTTINGDOWN SHUTDOWN ERROR

In addition, the class supports 'wait_until(state)' and 'wait_while(state)' methods, which, when called from a different process, block until the state has been entered or exited, respectively.

The 'RUNNING_CONDITIONS' list is a list of states that a thread may have if it is started/starting. The 'STOPPING_CONDITIONS' list is a list of states that a thread may have if it is stopped/stopping. The 'INACTIVE_CONDITIONS' list is a list of states that a thread may have when it is not active

class `decisionengine.framework.taskmanager.ProcessingState.`**`ProcessingState`**(*state=<State.BOOT: 0>*)

Bases: `object`

This object tracks the state of a process.

A number of convience wrappers are provided.

Additionally you may use the *.lock* attribute for *with* block to lock the state during specific operations.

`get`()

This function is a minimally locking check to fetch the state.

`has_value`(*state*)

`inactive`()

`property lock`

`probably_running`()

`set`(*state*)

This function will lock (and possibly block) to ensure a consistent change to the state value.

This function can be blocked using the *.lock* to force state sync between threads if need be.

`should_stop`()

`wait_until`(*state*, *timeout=None*)

`wait_while`(*state*, *timeout=None*)

class `decisionengine.framework.taskmanager.ProcessingState.`**`State`**(*value*)

Bases: `enum.Enum`

An enumeration.

`ACTIVE = 2`

`BOOT = 0`

`ERROR = 7`

`IDLE = 1`

`OFFLINE = 6`

SHUTDOWN = 5

SHUTTINGDOWN = 4

STEADY = 3

decisionengine.framework.taskmanager.TaskManager module

Task Manager

class decisionengine.framework.taskmanager.TaskManager.**Channel**(*channel_dict*)

Bases: object

Decision Channel. Instantiates workers according to channel configuration

class decisionengine.framework.taskmanager.TaskManager.**TaskManager**(*name*, *generation_id*, *channel_dict*, *global_config*)

Bases: object

Task Manager

data_block_put(*data*, *header*, *data_block*)

Put data into data block

Parameters

- **data** (dict) – key, value pairs
- **header** (Header) – data header
- **data_block** (DataBlock) – data block

decision_cycle()

Decision cycle to be run periodically (by trigger)

do_backup()

Duplicate current data block and return its copy

Return type DataBlock

get_consumes()

get_loglevel()

get_produces()

get_state()

get_state_name()

get_state_value()

run()

Task Manager main loop

run_logic_engine(*data_block=None*)

Run Logic Engine.

Parameters **data_block** (DataBlock) – data block

run_publishers(*actions*, *facts*, *data_block=None*)

Run Publishers in main process.

Parameters **data_block** (DataBlock) – data block

run_source(*src*)
Get the data from source and put it into the data block

Parameters **src** (*Worker*) – source Worker

run_transform(*transform*, *data_block*)
Run a transform

Parameters

- **transform** (*Worker*) – source Worker
- **data_block** (DataBlock) – data block

run_transforms(*data_block=None*)
Run transforms. So far in main process.

Parameters **data_block** (DataBlock) – data block

set_loglevel_value(*log_level*)
Assumes log_level is a string corresponding to the supported logging-module levels.

start_sources(*data_block=None*)
Start sources, each in a separate thread

Parameters **data_block** (DataBlock) – data block

take_offline(*current_data_block*)
offline and stop task manager

wait_for_all(*events_done*)
Wait for all sources or transforms to finish

Parameters **events_done** (list) – list of events to wait for

wait_for_any(*events_done*)
Wait for any sources to finish

Parameters **events_done** (list) – list of events to wait for

class decisionengine.framework.taskmanager.TaskManager.**Worker**(*conf_dict*, *base_class*)
Bases: object

Provides interface to loadable modules an events to sycronise execution

decisionengine.framework.taskmanager.TaskManager.**_create_module_instance**(*config_dict*, *base_class*)
Create instance of dynamically loaded module

decisionengine.framework.taskmanager.TaskManager.**_find_only_one_subclass**(*module*, *base_class*)
Search through module looking for only one subclass of the supplied base_class

decisionengine.framework.taskmanager.TaskManager.**_make_workers_for**(*configs*, *base_class*)

decisionengine.framework.taskmanager.module_graph module

Ensure no circularities in produces and consumes.

`decisionengine.framework.taskmanager.module_graph.`**`_consumed_products`**(**worker_lists*)

`decisionengine.framework.taskmanager.module_graph.`**`_produced_products`**(**worker_lists*)

`decisionengine.framework.taskmanager.module_graph.`**`ensure_no_circularities`**(*sources*, *transforms*, *publishers*)

DOCSTRING

Module contents

decisionengine.framework.tests package

Submodules

decisionengine.framework.tests.ABTransform module

class `decisionengine.framework.tests.ABTransform.`**`ABTransform`**(*module_parameters*, **args*, ***kwargs*)

Bases: *decisionengine.framework.modules.Transform.Transform*

`_consumes = {'B': None}`

`_produces = {'A': None}`

decisionengine.framework.tests.BATransform module

class `decisionengine.framework.tests.BATransform.`**`BATransform`**(*module_parameters*, **args*, ***kwargs*)

Bases: *decisionengine.framework.modules.Transform.Transform*

`_consumes = {'A': None}`

`_produces = {'B': None}`

decisionengine.framework.tests.FailingPublisher module

class `decisionengine.framework.tests.FailingPublisher.`**`FailingPublisher`**(*module_parameters*, **args*, ***kwargs*)

Bases: *decisionengine.framework.modules.Publisher.Publisher*

`_consumes = {'bar':  None}`

`publish`(*data_block*)

decisionengine.framework.tests.FailingSourceNOP module

class decisionengine.framework.tests.FailingSourceNOP.**SourceWithMissingProduces**(*set_of_parameters*)
Bases: *decisionengine.framework.modules.Source.Source*

decisionengine.framework.tests.FailingSourceProxy module

class decisionengine.framework.tests.FailingSourceProxy.**FailingSourceProxy**(*config*)
Bases: *decisionengine.framework.modules.SourceProxy.SourceProxy*

acquire()
Overrides Source class method

decisionengine.framework.tests.ModuleProgramOptions module

class decisionengine.framework.tests.ModuleProgramOptions.**AcquireWithConfig**(*name*)
Bases: object

test(*byte_str*, *expected_stderr=''*)

class decisionengine.framework.tests.ModuleProgramOptions.**AcquireWithSampleConfig**(*name*)
Bases: object

test()

class decisionengine.framework.tests.ModuleProgramOptions.**ConfigTemplate**(*name*)
Bases: object

test(*has_comments=False*)

class decisionengine.framework.tests.ModuleProgramOptions.**Describe**(*name*)
Bases: object

test(*consumes=None*, *produces=None*)

class decisionengine.framework.tests.ModuleProgramOptions.**DescribeAlias**(*alias*, *original*)
Bases: object

test()

class decisionengine.framework.tests.ModuleProgramOptions.**Help**(*name*)
Bases: object

test(*has_sample_config=False*)

decisionengine.framework.tests.ModuleProgramOptions.**_expected_acquire_result**(*name*, *config_file=None*, *multiplier=1*)

decisionengine.framework.tests.ModuleProgramOptions.**_expected_config_template**(*name*)

decisionengine.framework.tests.ModuleProgramOptions.**_expected_config_template_with_comments**(*name*)

decisionengine.framework.tests.ModuleProgramOptions.**_expected_help**(*name*)

decisionengine.framework.tests.ModuleProgramOptions.**_expected_source_help**(*name*, *has_sample_config=False*)

decisionengine.framework.tests.ModuleProgramOptions.**_normalize**(*string*)

decisionengine.framework.tests.ModuleProgramOptions.**_run_as_main**(*name*, **program_options*)

decisionengine.framework.tests.PublisherNOP module

class decisionengine.framework.tests.PublisherNOP.**PublisherNOP**(*module_parameters*, **args*, ***kwargs*)

Bases: *decisionengine.framework.modules.Publisher.Publisher*

_consumes = {'bar': <class 'pandas.core.frame.DataFrame'>}

publish(*data_block*)

decisionengine.framework.tests.PublisherWithMissingConsumes module

class decisionengine.framework.tests.PublisherWithMissingConsumes.**PublisherWithMissingConsumes**(*set_of_par*

Bases: *decisionengine.framework.modules.Publisher.Publisher*

decisionengine.framework.tests.SourceAlias module

decisionengine.framework.tests.SourceNOP module

class decisionengine.framework.tests.SourceNOP.**SourceNOP**(*config*)

Bases: *decisionengine.framework.modules.Source.Source*

_produces = {'foo': <class 'pandas.core.frame.DataFrame'>}

acquire()

decisionengine.framework.tests.SourceWithSampleConfigNOP module

class decisionengine.framework.tests.SourceWithSampleConfigNOP.**SourceWithSampleConfigNOP**(*config*)

Bases: *decisionengine.framework.modules.Source.Source*

_produces = {'foo': <class 'pandas.core.frame.DataFrame'>}

_supported_config = {'multiplier': (<class 'int'>, None, None)}

acquire()

decisionengine.framework.tests.SupportsConfigPublisher module

class decisionengine.framework.tests.SupportsConfigPublisher.**SupportsConfig**(*set_of_parameters*)

Bases: *decisionengine.framework.modules.Publisher.Publisher*

_supported_config = {'comment': (<class 'str'>, None, 'Single-line comment'), 'comment_with_nl': (<class 'str'>, None, 'Comment with newline\n'), 'convert_to': (<class 'int'>, 3, None), 'default_only': (<class 'float'>, 2.5, None), 'no_type': (None, None, None), 'only_type': (<class 'int'>, None, None)}

decisionengine.framework.tests.TransformNOP module

class decisionengine.framework.tests.TransformNOP.**TransformNOP**(*module_parameters*, **args*, ***kwargs*)

Bases: *decisionengine.framework.modules.Transform.Transform*

_consumes = {'foo': <class 'pandas.core.frame.DataFrame'>}

_produces = {'bar': <class 'pandas.core.frame.DataFrame'>}

transform(*data_block*)

decisionengine.framework.tests.TransformWithMissingProducesConsumes module

class decisionengine.framework.tests.TransformWithMissingProducesConsumes.**TransformWithMissingProducesC**

Bases: *decisionengine.framework.modules.Transform.Transform*

transform(*data_block*)

decisionengine.framework.tests.WorkingSourceProxy module

class decisionengine.framework.tests.WorkingSourceProxy.**WorkingSourceProxy**(*config*)

Bases: *decisionengine.framework.modules.SourceProxy.SourceProxy*

acquire()

Overrides Source class method

decisionengine.framework.tests.fixtures module

defaults for pytest

decisionengine.framework.tests.fixtures.**DEServer**(*conf_path=None*, *conf_override=None*, *channel_conf_path=None*, *channel_conf_override=None*, *host='127.0.0.1'*, *port=None*, *pg_prog_name='PG_PROG'*, *pg_db_conn_name='DE_DB'*)

A DE Server using a private database

decisionengine.framework.tests.fixtures.**DE_DB**(*request: _pytest.fixtures.FixtureRequest*) → psycopg2.extensions.connection

Fixture factory for PostgreSQL.

Parameters **request** (*FixtureRequest*) – fixture request object

Returns postgresql client

decisionengine.framework.tests.fixtures.**PG_PROG**(*request: _pytest.fixtures.FixtureRequest*, *tmpdir_factory: _pytest.tmpdir.TempdirFactory*) → pytest_postgresql.executor.PostgreSQLExecutor

Process fixture for PostgreSQL.

Parameters **request** (*FixtureRequest*) – fixture request object

Return type pytest_dbfixtures.executors.TCPExecutor

Returns tcp executor

decisionengine.framework.tests.test_client_errors module

Fixture based DE Server tests of the sample config

`decisionengine.framework.tests.test_client_errors.`**`test_client_cannot_wait_on_bad_state`**`(`*deserver*`)`
Verify wait is for a valid state

decisionengine.framework.tests.test_client_server module

Fixture based DE Server for the de-client tests

`decisionengine.framework.tests.test_client_server.`**`test_client_print_product`**`(`*deserver*`)`

`decisionengine.framework.tests.test_client_server.`**`test_client_status_msg_to_stdout`**`(`*deserver*`)`
Make sure the actuall client console call goes to stdout

decisionengine.framework.tests.test_defaults module

Fixture based DE Server tests of the sample config

`decisionengine.framework.tests.test_defaults.`**`test_client_can_get_de_server_show_channel_logger_level`**`(`*des*
Verify unknown channel has NOTSET

`decisionengine.framework.tests.test_defaults.`**`test_client_de_config_is_json`**`(`*deserver*`)`
Verify config can be fetched in json format

`decisionengine.framework.tests.test_defaults.`**`test_global_channel_log_level_in_config`**`(`*deserver*`)`
Verify global_channel_log_level setting exists

decisionengine.framework.tests.test_module_program_options module

`decisionengine.framework.tests.test_module_program_options.`**`test_acquire_for_sources`**`()`

`decisionengine.framework.tests.test_module_program_options.`**`test_config_templates`**`()`

`decisionengine.framework.tests.test_module_program_options.`**`test_descriptions`**`()`

`decisionengine.framework.tests.test_module_program_options.`**`test_help`**`()`

`decisionengine.framework.tests.test_module_program_options.`**`test_module_alias`**`()`

decisionengine.framework.tests.test_query_tool_server module

Fixture based DE Server for the de-query-tool tests

`decisionengine.framework.tests.test_query_tool_server.`**`test_query_tool_csv`**`(`*deserver*`)`

`decisionengine.framework.tests.test_query_tool_server.`**`test_query_tool_default`**`(`*deserver*`)`

`decisionengine.framework.tests.test_query_tool_server.`**`test_query_tool_invalid_product`**`(`*deserver*`)`

`decisionengine.framework.tests.test_query_tool_server.`**`test_query_tool_json`**`(`*deserver*`)`

`decisionengine.framework.tests.test_query_tool_server.`**`test_query_tool_since`**`(`*deserver*`)`

decisionengine.framework.tests.test_reaper module

Fixture based DE Server for the reaper tests

`decisionengine.framework.tests.test_reaper.`**`test_client_can_get_de_server_reaper_start_delay`**(*deserver*)
Verify reaper can start with delay

`decisionengine.framework.tests.test_reaper.`**`test_client_can_get_de_server_reaper_status`**(*deserver*)
Verify reaper status

`decisionengine.framework.tests.test_reaper.`**`test_client_can_get_de_server_reaper_stop`**(*deserver*)
Verify reaper can stop

decisionengine.framework.tests.test_restart_channel module

`decisionengine.framework.tests.test_restart_channel.`**`deserver_mock_data_block`**(*mock_data_block*)

`decisionengine.framework.tests.test_restart_channel.`**`test_restart_channel`**(*deserver_mock_data_block*)

decisionengine.framework.tests.test_sample_config module

Fixture based DE Server tests of the defaults

`decisionengine.framework.tests.test_sample_config.`**`test_client_can_double_set_de_server_channel_log_level`**
Verify set log level to current level isn't an error

`decisionengine.framework.tests.test_sample_config.`**`test_client_can_get_de_server_channel_config`**(*deserver*)
Verify config has expected items

`decisionengine.framework.tests.test_sample_config.`**`test_client_can_get_de_server_channel_log_level`**(*deserve*
Verify can fetch log level for a channel

`decisionengine.framework.tests.test_sample_config.`**`test_client_can_get_de_server_show_config`**(*deserver*)
Verify config has expected items

`decisionengine.framework.tests.test_sample_config.`**`test_client_can_get_de_server_show_logger_level`**(*deserve*
Verify can fetch log level

`decisionengine.framework.tests.test_sample_config.`**`test_client_can_get_de_server_status`**(*deserver*)
Verify channel enters stable state

`decisionengine.framework.tests.test_sample_config.`**`test_client_can_get_products`**(*deserver*)
Verify client can get channel products

`decisionengine.framework.tests.test_sample_config.`**`test_client_can_get_products_no_channels`**(*deserver*)
Verify client can get channel products even when none are run

`decisionengine.framework.tests.test_sample_config.`**`test_client_can_kill_one_channel`**(*deserver*)
Verify client can kill a single channel

`decisionengine.framework.tests.test_sample_config.`**`test_client_can_kill_one_channel_force`**(*deserver*)
Verify client can kill a single channel with force

`decisionengine.framework.tests.test_sample_config.`**`test_client_can_kill_one_channel_timeout`**(*deserver*)
Verify client can kill a single channel with timeout

`decisionengine.framework.tests.test_sample_config.`**`test_client_can_set_de_server_channel_log_level`**(*deserve*
Verify set log level for a channel

`decisionengine.framework.tests.test_sample_config.`**`test_client_can_start_all_channel`**(*deserver*)
Verify client can start all channel

`decisionengine.framework.tests.test_sample_config.`**`test_client_can_start_one_channel`**(*deserver*)
Verify client can start a single channel

`decisionengine.framework.tests.test_sample_config.`**`test_client_can_stop_channels`**(*deserver*)
Verify client can stop channels

`decisionengine.framework.tests.test_sample_config.`**`test_client_can_stop_one_channel`**(*deserver*)
Verify client can stop a single channel

`decisionengine.framework.tests.test_sample_config.`**`test_client_can_stop_server`**(*deserver*)
Verify de-client can run –stop

`decisionengine.framework.tests.test_sample_config.`**`test_client_cannot_double_start`**(*deserver*)
Verify client cannot double start channels

`decisionengine.framework.tests.test_sample_config.`**`test_client_get_channel_log_fails_cleanly`**(*deserver*)
Verify graceful fail on bogus channel

`decisionengine.framework.tests.test_sample_config.`**`test_client_get_non_real_channel`**(*deserver*)
Verify config for missing channel does what it should

`decisionengine.framework.tests.test_sample_config.`**`test_client_set_channel_log_fails_cleanly`**(*deserver*)
Verify graceful fail on bogus channel

`decisionengine.framework.tests.test_sample_config.`**`test_client_start_non_real_channel`**(*deserver*)
Verify start for missing channel does what it should

`decisionengine.framework.tests.test_sample_config.`**`test_client_stop_non_real_channel`**(*deserver*)
Verify stop for missing channel does what it should

decisionengine.framework.tests.test_source_proxy module

Fixture based tests of the SourceProxy module.

`decisionengine.framework.tests.test_source_proxy.`**`test_stop_failing_source_proxy`**(*deserver_fail*)

`decisionengine.framework.tests.test_source_proxy.`**`test_working_source_proxy`**(*deserver*)

decisionengine.framework.tests.test_start_with_bad_channels module

Fixture based DE Server tests of invalid channel configs

`decisionengine.framework.tests.test_start_with_bad_channels.`**`_consumes_not_subset`**(*test_str*)

`decisionengine.framework.tests.test_start_with_bad_channels.`**`_expected_circularity`**(*test_str*)

`decisionengine.framework.tests.test_start_with_bad_channels.`**`_missing_consumes`**(*name*)

`decisionengine.framework.tests.test_start_with_bad_channels.`**`_missing_produces`**(*name*)

`decisionengine.framework.tests.test_start_with_bad_channels.`**`test_client_can_get_products_no_channels`**(*des*
cap

Verify client can get channel products even when none are run

decisionengine.framework.tests.test_start_with_no_channels module

Fixture based DE Server tests of the server without channels, then with them

`decisionengine.framework.tests.test_start_with_no_channels.`**`deserver_mock_data_block`**(*mock_data_block*)

`decisionengine.framework.tests.test_start_with_no_channels.`**`test_start_from_nothing`**(*deserver_mock_data_bloc*

Module contents

decisionengine.framework.util package

Submodules

decisionengine.framework.util.fs module

`decisionengine.framework.util.fs.`**`files_with_extensions`**(*dir_path*, **extensions*)

Return all files in dir_path that match the provided extensions.

If no extensions are given, then all files in dir_path are returned.

Results are sorted by channel name to ensure stable output.

decisionengine.framework.util.reaper module

A stand-alone script purges data in database older than specified in configuration. Configuration file has to have this bit added:

```
{
        "dataspace" [{ "retention_interval_in_days"][365,]
            "datasource" : { ... }
        }
    }
```

Can be used in a cron job.

`decisionengine.framework.util.reaper.`**`main`**()

decisionengine.framework.util.sockets module

`decisionengine.framework.util.sockets.`**`get_random_port`**()

decisionengine.framework.util.subclasses module

`decisionengine.framework.util.subclasses.`**`_derived_class`**(*cls*, *base_class*)
Only matches subclasses that are not equal to the base class.

`decisionengine.framework.util.subclasses.`**`all_subclasses`**(*module*, *base_class*)
Return all of a module's subclasses of the given base class.

Module contents

Submodules

decisionengine.framework.about module

PEP-0396 provides instructions for providing module versions While we are at it, add a few other useful bits

decisionengine.framework.version module

Module contents

decisionengine.tests package

Submodules

decisionengine.tests.test_framework_package module

Make sure decisionengine.framework is a valid python package

`decisionengine.tests.test_framework_package.`**`test_can_import`**()

Module contents

Module contents

4.2 Indices and tables

- genindex
- modindex
- search

CHAPTER

FIVE

INDICES AND TABLES

- genindex
- modindex
- search

PYTHON MODULE INDEX

d

decisionengine, 65
decisionengine.framework, 65
decisionengine.framework.about, 65
decisionengine.framework.config, 27
decisionengine.framework.config.ChannelConfigHandler, 25
decisionengine.framework.config.policies, 26
decisionengine.framework.config.tests, 25
decisionengine.framework.config.tests.test_config, 23
decisionengine.framework.config.tests.test_policies, 24
decisionengine.framework.config.tests.test_validconfig, 24
decisionengine.framework.config.ValidConfig, 25
decisionengine.framework.dataspace, 39
decisionengine.framework.dataspace.datablock, 33
decisionengine.framework.dataspace.datasource, 35
decisionengine.framework.dataspace.datasources, 31
decisionengine.framework.dataspace.datasources.postgresql, 28
decisionengine.framework.dataspace.datasources.tests, 28
decisionengine.framework.dataspace.datasources.tests.fixtures, 27
decisionengine.framework.dataspace.datasources.tests.test_postgresql, 27
decisionengine.framework.dataspace.dataspace, 38
decisionengine.framework.dataspace.maintain, 39
decisionengine.framework.dataspace.tests, 33
decisionengine.framework.dataspace.tests.test_datablock, 32
decisionengine.framework.dataspace.tests.test_datasource, 33
decisionengine.framework.dataspace.tests.test_dataspace, 33
decisionengine.framework.dataspace.tests.test_Reaper, 31
decisionengine.framework.engine, 45
decisionengine.framework.engine.de_client, 44
decisionengine.framework.engine.de_query_tool, 44
decisionengine.framework.engine.DecisionEngine, 41
decisionengine.framework.engine.tests, 41
decisionengine.framework.engine.tests.fixtures, 39
decisionengine.framework.engine.tests.test_client_only, 40
decisionengine.framework.engine.tests.test_query_tool_only 40
decisionengine.framework.engine.tests.test_startup, 41
decisionengine.framework.engine.Workers, 43
decisionengine.framework.logicengine, 49
decisionengine.framework.logicengine.BooleanExpression, 47
decisionengine.framework.logicengine.FactLookup, 47
decisionengine.framework.logicengine.LogicEngine, 48
decisionengine.framework.logicengine.Rule, 48
decisionengine.framework.logicengine.RuleEngine, 49
decisionengine.framework.logicengine.tests, 47
decisionengine.framework.logicengine.tests.test_cascaded_r 45
decisionengine.framework.logicengine.tests.test_constructi 45
decisionengine.framework.logicengine.tests.test_duplicate_ 45
decisionengine.framework.logicengine.tests.test_facts, 45
decisionengine.framework.logicengine.tests.test_fail_on_er 46
decisionengine.framework.logicengine.tests.test_pandas_fac

46
decisionengine.framework.logicengine.tests.test_cli, 46
decisionengine.framework.logicengine.tests.test_simple_configuration, 46
decisionengine.framework.modules, 54
decisionengine.framework.modules.de_logger, 52
decisionengine.framework.modules.describe, 53
decisionengine.framework.modules.LogicEngine, 50
decisionengine.framework.modules.Module, 51
decisionengine.framework.modules.print_description, 53
decisionengine.framework.modules.Publisher, 51
decisionengine.framework.modules.Source, 51
decisionengine.framework.modules.SourceProxy, 52
decisionengine.framework.modules.tests, 50
decisionengine.framework.modules.tests.test_decorator, 50
decisionengine.framework.modules.tests.test_LogicEngine, 49
decisionengine.framework.modules.tests.test_Module, 49
decisionengine.framework.modules.tests.test_modules_decorators, 50
decisionengine.framework.modules.tests.test_Publisher, 49
decisionengine.framework.modules.tests.test_Source, 50
decisionengine.framework.modules.tests.test_Transform, 50
decisionengine.framework.modules.Transform, 52
decisionengine.framework.taskmanager, 57
decisionengine.framework.taskmanager.module_graph, 57
decisionengine.framework.taskmanager.ProcessingState, 54
decisionengine.framework.taskmanager.TaskManager, 55
decisionengine.framework.tests, 64
decisionengine.framework.tests.ABTransform, 57
decisionengine.framework.tests.BATransform, 57
decisionengine.framework.tests.FailingPublisher, 57
decisionengine.framework.tests.FailingSourceNOP, 58
decisionengine.framework.tests.FailingSourceProxy, 58
decisionengine.framework.tests.fixtures, 60
decisionengine.framework.tests.ModuleProgramOptions, 58
decisionengine.framework.tests.PublisherNOP, 59
decisionengine.framework.tests.PublisherWithMissingConsume 59
decisionengine.framework.tests.SourceAlias, 59
decisionengine.framework.tests.SourceNOP, 59
decisionengine.framework.tests.SourceWithSampleConfigNOP, 59
decisionengine.framework.tests.SupportsConfigPublisher, 59
decisionengine.framework.tests.test_client_errors, 61
decisionengine.framework.tests.test_client_server, 61
decisionengine.framework.tests.test_defaults, 61
decisionengine.framework.tests.test_module_program_options 61
decisionengine.framework.tests.test_query_tool_server, 61
decisionengine.framework.tests.test_reaper, 62
decisionengine.framework.tests.test_restart_channel, 62
decisionengine.framework.tests.test_sample_config, 62
decisionengine.framework.tests.test_source_proxy, 63
decisionengine.framework.tests.test_start_with_bad_channel 63
decisionengine.framework.tests.test_start_with_no_channels 64
decisionengine.framework.tests.TransformNOP, 60
decisionengine.framework.tests.TransformWithMissingProduce 60
decisionengine.framework.tests.WorkingSourceProxy, 60
decisionengine.framework.util, 65
decisionengine.framework.util.fs, 64
decisionengine.framework.util.reaper, 64
decisionengine.framework.util.sockets, 64
decisionengine.framework.util.subclasses, 65
decisionengine.framework.version, 65
decisionengine.tests, 65
decisionengine.tests.test_framework_package, 65

INDEX

Symbols

__query() (*decisionengine.framework.dataspace.datasources.postgresql.Postgresql method*), 28
_abc_impl (*decisionengine.framework.config.ValidConfig.ValidConfig attribute*), 25
_abc_impl (*decisionengine.framework.dataspace.datablock.Header attribute*), 34
_abc_impl (*decisionengine.framework.dataspace.datablock.Metadata attribute*), 35
_abc_impl (*decisionengine.framework.dataspace.datasource.DataSource attribute*), 35
_abc_impl (*decisionengine.framework.dataspace.datasources.postgresql.Postgresql attribute*), 28
_channel_config_dir() (*in module decisionengine.framework.config.tests.test_config*), 23
_channel_preamble() (*in module decisionengine.framework.engine.DecisionEngine*), 42
_check_keys() (*in module decisionengine.framework.config.ChannelConfigHandler*), 25
_check_override() (*in module decisionengine.framework.engine.tests.test_startup*), 41
_config_from_file() (*in module decisionengine.framework.config.ValidConfig*), 26
_consumed_products() (*in module decisionengine.framework.taskmanager.module_graph*), 57
_consumes (*decisionengine.framework.modules.Publisher.Publisher attribute*), 51
_consumes (*decisionengine.framework.modules.Transform.Transform attribute*), 52
_consumes (*decisionengine.framework.tests.ABTransform.ABTransform attribute*), 57
_consumes (*decisionengine.framework.tests.BATransform.BATransform attribute*), 57
_consumes (*decisionengine.framework.tests.FailingPublisher.FailingPublisher attribute*), 57
_consumes (*decisionengine.framework.tests.PublisherNOP.PublisherNOP attribute*), 59
_consumes (*decisionengine.framework.tests.TransformNOP.TransformNOP attribute*), 60
_consumes_not_subset() (*in module decisionengine.framework.tests.test_start_with_bad_channels*), 63
_convert_to_json() (*in module decisionengine.framework.config.ValidConfig*), 26
_create_de_server() (*in module decisionengine.framework.engine.DecisionEngine*), 43
_create_facts_dataframe() (*decisionengine.framework.logicengine.LogicEngine.LogicEngine method*), 48
_create_module_instance() (*in module decisionengine.framework.taskmanager.TaskManager*), 56
_dataframe_to_column_names() (*decisionengine.framework.engine.DecisionEngine.DecisionEngine method*), 41
_dataframe_to_csv() (*decisionengine.framework.engine.DecisionEngine.DecisionEngine method*), 41
_dataframe_to_json() (*decisionengine.framework.engine.DecisionEngine.DecisionEngine method*), 41
_dataframe_to_table() (*decisionengine.framework.engine.DecisionEngine.DecisionEngine method*), 41
_dataframe_to_vertical_tables() (*decisionengine.framework.engine.DecisionEngine.DecisionEngine method*), 41
_delete() (*decisionengine.framework.dataspace.datasources.postgresql.Postgresql method*), 28
_derived_class() (*in module decisionengine.framework.util.subclasses*), 65
_dispatch() (*decisionengine.framework.engine.DecisionEngine.DecisionEngine method*), 41
_ds (*decisionengine.framework.dataspace.dataspace.DataSourceLoader attribute*), 38

_expected_acquire_result() (*in module decisionengine.framework.tests.ModuleProgramOptions*), 58
_expected_circularity() (*in module decisionengine.framework.tests.test_start_with_bad_channels*), 63
_expected_config_template() (*in module decisionengine.framework.tests.ModuleProgramOptions*), 58
_expected_config_template_with_comments() (*in module decisionengine.framework.tests.ModuleProgramOptions*), 58
_expected_help() (*in module decisionengine.framework.tests.ModuleProgramOptions*), 58
_expected_source_help() (*in module decisionengine.framework.tests.ModuleProgramOptions*), 58
_find_only_one_subclass() (*in module decisionengine.framework.taskmanager.TaskManager*), 56
_get_data() (*decisionengine.framework.modules.SourceProxy.SourceProxy method*), 52
_get_de_conf_manager() (*in module decisionengine.framework.engine.DecisionEngine*), 43
_get_global_config() (*in module decisionengine.framework.engine.DecisionEngine*), 43
_global_config_file() (*in module decisionengine.framework.config.tests.test_config*), 23
_global_config_file() (*in module decisionengine.framework.config.tests.test_validconfig*), 24
_insert() (*decisionengine.framework.dataspace.datablock.DataBlock method*), 33
_insert() (*decisionengine.framework.dataspace.datasources.postgresql.Postgresql method*), 28
_insert_returning_result() (*decisionengine.framework.dataspace.datasources.postgresql.Postgresql method*), 28
_instances (*decisionengine.framework.dataspace.dataspace.Singleton attribute*), 39
_load_channel() (*decisionengine.framework.config.ChannelConfigHandler.ChannelConfigHandler method*), 25
_make_de_logger() (*in module decisionengine.framework.config.ChannelConfigHandler*), 25
_make_workers_for() (*in module decisionengine.framework.taskmanager.TaskManager*), 56
_missing_consumes() (*in module decisionengine.framework.tests.test_start_with_bad_channels*), 63
_missing_produces() (*in module decisionengine.framework.tests.test_start_with_bad_channels*), 63
_normalize() (*in module decisionengine.framework.tests.ModuleProgramOptions*), 58
_par_default() (*in module decisionengine.framework.modules.describe*), 53
_par_type() (*in module decisionengine.framework.modules.describe*), 53
_print_comment() (*in module decisionengine.framework.modules.print_description*), 53
_print_type() (*in module decisionengine.framework.modules.print_description*), 53
_print_value() (*in module decisionengine.framework.modules.print_description*), 53
_produced_products() (*in module decisionengine.framework.taskmanager.module_graph*), 57
_produces (*decisionengine.framework.modules.Source.Source attribute*), 51
_produces (*decisionengine.framework.modules.Transform.Transform attribute*), 52
_produces (*decisionengine.framework.tests.ABTransform.ABTransform attribute*), 57
_produces (*decisionengine.framework.tests.BATransform.BATransform attribute*), 57
_produces (*decisionengine.framework.tests.SourceNOP.SourceNOP attribute*), 59
_produces (*decisionengine.framework.tests.SourceWithSampleConfigNOP.SourceWithSampleConfigNOP attribute*), 59
_produces (*decisionengine.framework.tests.TransformNOP.TransformNOP attribute*), 60
_reaper_loop() (*decisionengine.framework.dataspace.maintain.Reaper method*), 39
_remove() (*decisionengine.framework.dataspace.datasources.postgresql.Postgresql method*), 28
_run_as_main() (*in module decisionengine.framework.tests.ModuleProgramOptions*), 58
_select() (*decisionengine.framework.dataspace.datasources.postgresql.Postgresql method*), 28
_select_dictresult() (*decisionengine.framework.dataspace.datasources.postgresql.Postgresql method*), 28
_select_getresult() (*decisio-*

nengine.framework.dataspace.datasources.postgresql.Postgresql method), 28
_select_tuple() (*decisionengine.framework.dataspace.datasources.postgresql.Postgresql method*), 28
_setitem() (*decisionengine.framework.dataspace.datablock.DataBlock method*), 33
_spec_from_file_name() (*in module decisionengine.framework.modules.print_description*), 53
_start_de_server() (*in module decisionengine.framework.engine.DecisionEngine*), 43
_supported_config (*decisionengine.framework.modules.SourceProxy.SourceProxy attribute*), 52
_supported_config (*decisionengine.framework.tests.SourceWithSampleConfigNOP.SourceWithSampleConfigNOP attribute*), 59
_supported_config (*decisionengine.framework.tests.SupportsConfigPublisher.SupportsConfig attribute*), 59
_tables_created (*decisionengine.framework.dataspace.dataspace.DataSpace attribute*), 38
_update() (*decisionengine.framework.dataspace.datablock.DataBlock method*), 33
_update() (*decisionengine.framework.dataspace.datasources.postgresql.Postgresql method*), 28
_update_channel_states() (*decisionengine.framework.engine.Workers.Workers method*), 44
_update_returning_result() (*decisionengine.framework.dataspace.datasources.postgresql.Postgresql method*), 28

A

ABTransform (*class in decisionengine.framework.tests.ABTransform*), 57
access() (*decisionengine.framework.engine.Workers.Workers method*), 44
acquire() (*decisionengine.framework.modules.Source.Source method*), 51
acquire() (*decisionengine.framework.modules.SourceProxy.SourceProxy method*), 52
acquire() (*decisionengine.framework.tests.FailingSourceProxy.FailingSourceProxy method*), 58
acquire() (*decisionengine.framework.tests.SourceNOP.SourceNOP method*), 59
acquire() (*decisionengine.framework.tests.SourceWithSampleConfigNOP.SourceWithSampleConfigNOP method*), 59
acquire() (*decisionengine.framework.tests.WorkingSourceProxy.WorkingSourceProxy method*), 60
AcquireWithConfig (*class in decisionengine.framework.tests.ModuleProgramOptions*), 58
AcquireWithSampleConfig (*class in decisionengine.framework.tests.ModuleProgramOptions*), 58
ACTIVE (*decisionengine.framework.taskmanager.ProcessingState.State attribute*), 54
all_subclasses() (*in module decisionengine.framework.util.subclasses*), 65

B

BATransform (*class in decisionengine.framework.tests.BATransform*), 57
block_until() (*decisionengine.framework.engine.DecisionEngine.DecisionEngine method*), 41
block_while() (*decisionengine.framework.engine.DecisionEngine.DecisionEngine method*), 41
BooleanExpression (*class in decisionengine.framework.logicengine.BooleanExpression*), 47
BOOT (*decisionengine.framework.taskmanager.ProcessingState.State attribute*), 54

C

Channel (*class in decisionengine.framework.taskmanager.TaskManager*), 55
channel_config_dir() (*in module decisionengine.framework.config.policies*), 26
ChannelConfigHandler (*class in decisionengine.framework.config.ChannelConfigHandler*), 25
Clean (*decisionengine.framework.engine.DecisionEngine.StopState attribute*), 42
close() (*decisionengine.framework.dataspace.datasource.DataSource method*), 35
close() (*decisionengine.framework.dataspace.datasources.postgresql.Postgresql method*), 28
close() (*decisionengine.framework.dataspace.dataspace.DataSpace method*), 38
compress() (*in module decisionengine.framework.dataspace.datablock*), 33
ConfigTemplate (*class in decisionengine.framework.tests.ModuleProgramOptions*), 58
connect() (*decisionengine.framework.dataspace.datasource.DataSource method*), 35
connect() (*decisionengine.framework.dataspace.datasources.postgresql.Postgresql method*), 29

console_scripts_main() (*in module decisionengine.framework.engine.de_client*), 44
consumes() (*decisionengine.framework.logicengine.LogicEngine.LogicEngine method*), 48
consumes() (*in module decisionengine.framework.modules.Module*), 51
consumes() (*in module decisionengine.framework.modules.Publisher*), 51
consumes() (*in module decisionengine.framework.modules.Transform*), 52
create_datasource() (*decisionengine.framework.dataspace.dataspace.DataSourceLoader static method*), 38
create_parser() (*in module decisionengine.framework.engine.de_client*), 44
create_parser() (*in module decisionengine.framework.engine.de_query_tool*), 44
create_tables() (*decisionengine.framework.dataspace.datasource.DataSource method*), 35
create_tables() (*decisionengine.framework.dataspace.datasources.postgresql.PostgreSQL method*), 29

D

data() (*in module decisionengine.framework.dataspace.datasources.tests.test_postgresql*), 27
data() (*in module decisionengine.framework.dataspace.tests.test_datablock*), 32
data_block_put() (*decisionengine.framework.taskmanager.TaskManager.TaskManager method*), 55
DataBlock (*class in decisionengine.framework.dataspace.datablock*), 33
dataproduct() (*in module decisionengine.framework.dataspace.datasources.tests.test_postgresql*), 27
dataproduct_table (*decisionengine.framework.dataspace.datasource.DataSource attribute*), 35
DataSource (*class in decisionengine.framework.dataspace.datasource*), 35
datasource() (*in module decisionengine.framework.dataspace.datasources.tests.test_postgresql*), 27
datasource() (*in module decisionengine.framework.dataspace.tests.test_datablock*), 33
DataSourceLoader (*class in decisionengine.framework.dataspace.dataspace*), 38
DataSpace (*class in decisionengine.framework.dataspace.dataspace*), 38
DataSpaceConfigurationError, 38
DataSpaceConnectionError, 38
DataSpaceError, 38
DataSpaceExistsError, 38
dblock() (*in module decisionengine.framework.dataspace.tests.test_datablock*), 33
DE_DB() (*in module decisionengine.framework.engine.tests.fixtures*), 40
DE_DB() (*in module decisionengine.framework.tests.fixtures*), 60
decision_cycle() (*decisionengine.framework.taskmanager.TaskManager.TaskManager method*), 55
decisionengine
module, 65
DecisionEngine (*class in decisionengine.framework.engine.DecisionEngine*), 41
decisionengine.framework
module, 65
decisionengine.framework.about
module, 65
decisionengine.framework.config
module, 27
decisionengine.framework.config.ChannelConfigHandler
module, 25
decisionengine.framework.config.policies
module, 26
decisionengine.framework.config.tests
module, 25
decisionengine.framework.config.tests.test_config
module, 23
decisionengine.framework.config.tests.test_policies
module, 24
decisionengine.framework.config.tests.test_validconfig
module, 24
decisionengine.framework.config.ValidConfig
module, 25
decisionengine.framework.dataspace
module, 39
decisionengine.framework.dataspace.datablock
module, 33
decisionengine.framework.dataspace.datasource
module, 35
decisionengine.framework.dataspace.datasources

module, 31
decisionengine.framework.dataspace.datasources.postgresql
module, 28
decisionengine.framework.dataspace.datasources.sql
module, 28
decisionengine.framework.dataspace.datasources.tests.fixtures
module, 27
decisionengine.framework.dataspace.datasources.tests.postgresql
module, 27
decisionengine.framework.dataspace.dataspace
module, 38
decisionengine.framework.dataspace.maintain
module, 39
decisionengine.framework.dataspace.tests
module, 33
decisionengine.framework.dataspace.tests.test_datablock
module, 32
decisionengine.framework.dataspace.tests.test_datasource
module, 33
decisionengine.framework.dataspace.tests.test_dataspace
module, 33
decisionengine.framework.dataspace.tests.test_keepsi
module, 31
decisionengine.framework.engine
module, 45
decisionengine.framework.engine.de_client
module, 44
decisionengine.framework.engine.de_query_tool
module, 44
decisionengine.framework.engine.DecisionEngine
module, 41
decisionengine.framework.engine.tests
module, 41
decisionengine.framework.engine.tests.fixtures
module, 39
decisionengine.framework.engine.tests.test_client_info
module, 40
decisionengine.framework.engine.tests.test_query_tool
module, 40
decisionengine.framework.engine.tests.test_startup
module, 41
decisionengine.framework.engine.Workers
module, 43
decisionengine.framework.logicengine
module, 49
decisionengine.framework.logicengine.BooleanExpression
module, 47
decisionengine.framework.logicengine.FactLookup
module, 47
decisionengine.framework.logicengine.LogicEngine
module, 48
decisionengine.framework.logicengine.Rule
module, 48
decisionengine.framework.logicengine.RuleEngine
module, 49
decisionengine.framework.logicengine.tests
module, 47
decisionengine.framework.logicengine.tests.test_cascaded_r
module, 45
decisionengine.framework.logicengine.tests.test_constructi
module, 45
decisionengine.framework.logicengine.tests.test_duplicate_
module, 45
decisionengine.framework.logicengine.tests.test_facts
module, 45
decisionengine.framework.logicengine.tests.test_fail_on_er
module, 46
decisionengine.framework.logicengine.tests.test_pandas_fac
module, 46
decisionengine.framework.logicengine.tests.test_rule_with_
module, 46
decisionengine.framework.logicengine.tests.test_simple_cor
module, 46
decisionengine.framework.modules
module, 54
decisionengine.framework.modules.de_logger
module, 52
decisionengine.framework.modules.describe
module, 53
decisionengine.framework.modules.LogicEngine
module, 50
decisionengine.framework.modules.Module
module, 51
decisionengine.framework.modules.print_description
module, 53
decisionengine.framework.modules.Publisher
module, 51
decisionengine.framework.modules.Source
module, 51
decisionengine.framework.modules.SourceProxy
module, 52
decisionengine.framework.modules.tests
module, 50
decisionengine.framework.modules.tests.test_de_logger
module, 50
decisionengine.framework.modules.tests.test_LogicEngine
module, 49
decisionengine.framework.modules.tests.test_Module
module, 49
decisionengine.framework.modules.tests.test_module_decorat
module, 50
decisionengine.framework.modules.tests.test_Publisher
module, 49
decisionengine.framework.modules.tests.test_Source
module, 50
decisionengine.framework.modules.tests.test_Transform
module, 50
decisionengine.framework.modules.Transform

module, 52
decisionengine.framework.taskmanager
module, 57
decisionengine.framework.taskmanager.module_graph
module, 57
decisionengine.framework.taskmanager.ProcessingState
module, 54
decisionengine.framework.taskmanager.TaskManager
module, 55
decisionengine.framework.tests
module, 64
decisionengine.framework.tests.ABTransform
module, 57
decisionengine.framework.tests.BATransform
module, 57
decisionengine.framework.tests.FailingPublisher
module, 57
decisionengine.framework.tests.FailingSourceNOP
module, 58
decisionengine.framework.tests.FailingSourceProxy
module, 58
decisionengine.framework.tests.fixtures
module, 60
decisionengine.framework.tests.ModuleProgramOptions
module, 58
decisionengine.framework.tests.PublisherNOP
module, 59
decisionengine.framework.tests.PublisherWithMissingConsumes
module, 59
decisionengine.framework.tests.SourceAlias
module, 59
decisionengine.framework.tests.SourceNOP
module, 59
decisionengine.framework.tests.SourceWithSampleConfigNOP
module, 59
decisionengine.framework.tests.SupportsConfigPublisher
module, 59
decisionengine.framework.tests.test_client_errors
module, 61
decisionengine.framework.tests.test_client_server
module, 61
decisionengine.framework.tests.test_defaults
module, 61
decisionengine.framework.tests.test_module_program_options
module, 61
decisionengine.framework.tests.test_query_tool_server
module, 61
decisionengine.framework.tests.test_reaper
module, 62
decisionengine.framework.tests.test_restart_channel
module, 62
decisionengine.framework.tests.test_sample_config
module, 62
decisionengine.framework.tests.test_source_proxy
module, 63
decisionengine.framework.tests.test_start_with_bad_channel
module, 63
decisionengine.framework.tests.test_start_with_no_channels
module, 64
decisionengine.framework.tests.TransformNOP
module, 60
decisionengine.framework.tests.TransformWithMissingProduces
module, 60
decisionengine.framework.tests.WorkingSourceProxy
module, 60
decisionengine.framework.util
module, 65
decisionengine.framework.util.fs
module, 64
decisionengine.framework.util.reaper
module, 64
decisionengine.framework.util.sockets
module, 64
decisionengine.framework.util.subclasses
module, 65
decisionengine.framework.version
module, 65
decisionengine.tests
module, 65
decisionengine.tests.test_framework_package
module, 65
decisionCycles() (*in module decisionengine.framework.dataspace.datablock*), 35
default_data_lifetime (*decisionengine.framework.dataspace.datablock.Header attribute*), 34
delete() (*decisionengine.framework.dataspace.dataspace.DataSpace method*), 38
delete_data_older_than() (*decisionengine.framework.dataspace.datasource.DataSource method*), 35
delete_data_older_than() (*decisionengine.framework.dataspace.datasources.postgresql.Postgresql method*), 29
delete_data_older_than() (*decisionengine.framework.dataspace.tests.test_Reaper.MockSource method*), 31
Describe (*class in decisionengine.framework.tests.ModuleProgramOptions*), 58
describe() (*in module decisionengine.framework.modules.Publisher*), 51
describe() (*in module decisionengine.framework.modules.Source*), 51
describe() (*in module decisionengine.framework.modules.Transform*),

52
DescribeAlias (*class in decisionengine.framework.tests.ModuleProgramOptions*), 58
DEServer() (*in module decisionengine.framework.engine.tests.fixtures*), 39
DEServer() (*in module decisionengine.framework.tests.fixtures*), 60
deserver_mock_data_block() (*in module decisionengine.framework.tests.test_restart_channel*), 62
deserver_mock_data_block() (*in module decisionengine.framework.tests.test_start_with_no_channels*), 64
do_backup() (*decisionengine.framework.taskmanager.TaskManager.TaskManager method*), 55
dspace() (*in module decisionengine.framework.dataspace.tests.test_datablock*), 33
dump() (*decisionengine.framework.config.ValidConfig.ValidConfig method*), 26
duplicate() (*decisionengine.framework.dataspace.datablock.DataBlock method*), 33
duplicate_datablock() (*decisionengine.framework.dataspace.datasource.DataSource method*), 35
duplicate_datablock() (*decisionengine.framework.dataspace.datasources.postgresql.Postgresql method*), 29
duplicate_datablock() (*decisionengine.framework.dataspace.dataspace.DataSpace method*), 38

E

ensure_no_circularities() (*in module decisionengine.framework.taskmanager.module_graph*), 57
ERROR (*decisionengine.framework.taskmanager.ProcessingState.State attribute*), 54
evaluate() (*decisionengine.framework.logicengine.BooleanExpression.BooleanExpression method*), 47
evaluate() (*decisionengine.framework.logicengine.LogicEngine.LogicEngine method*), 48
evaluate() (*decisionengine.framework.logicengine.Rule.Rule method*), 48
evaluate() (*decisionengine.framework.modules.LogicEngine.LogicEngine.LogicEngine method*), 50
evaluate_facts() (*decisionengine.framework.logicengine.LogicEngine.LogicEngine method*), 48
execute() (*decisionengine.framework.logicengine.RuleEngine.RuleEngine method*), 49
execute_command_from_args() (*in module decisionengine.framework.engine.de_client*), 44
execute_command_from_args() (*in module decisionengine.framework.engine.de_query_tool*), 44

F

FactLookup (*class in decisionengine.framework.logicengine.FactLookup*), 47
FailingPublisher (*class in decisionengine.framework.tests.FailingPublisher*), 57
FailingSourceProxy (*class in decisionengine.framework.tests.FailingSourceProxy*), 58
files_with_extensions() (*in module decisionengine.framework.util.fs*), 64
fixtures() (*in module decisionengine.framework.dataspace.tests.test_datablock*), 33
function_name_from_call() (*in module decisionengine.framework.logicengine.BooleanExpression*), 47

G

generate_insert_query() (*in module decisionengine.framework.dataspace.datasources.postgresql*), 31
get() (*decisionengine.framework.dataspace.datablock.DataBlock method*), 33
get() (*decisionengine.framework.taskmanager.ProcessingState.ProcessingState method*), 54
get_channels() (*decisionengine.framework.config.ChannelConfigHandler.ChannelConfigHandler method*), 25
get_connection() (*decisionengine.framework.dataspace.datasources.postgresql.Postgresql method*), 29
get_consumes() (*decisionengine.framework.engine.Workers.Worker.Worker method*), 43
get_consumes() (*decisionengine.framework.taskmanager.TaskManager.TaskManager method*), 55
get_data_block() (*decisionengine.framework.modules.Module.Module method*), 51
get_datablock() (*decisionengine.framework.dataspace.datasource.DataSource method*), 36
get_datablock() (*decisionengine.framework.dataspace.datasources.postgresql.Postgresql

method), 29
get_dataproduct() (*decisionengine.framework.dataspace.datasource.DataSource method*), 36
get_dataproduct() (*decisionengine.framework.dataspace.datasources.postgresql.Postgresql method*), 29
get_dataproduct() (*decisionengine.framework.dataspace.dataspace.DataSpace method*), 38
get_dataproducts() (*decisionengine.framework.dataspace.datablock.DataBlock method*), 33
get_dataproducts() (*decisionengine.framework.dataspace.datasource.DataSource method*), 36
get_dataproducts() (*decisionengine.framework.dataspace.datasources.postgresql.Postgresql method*), 29
get_dataproducts() (*decisionengine.framework.dataspace.dataspace.DataSpace method*), 38
get_header() (*decisionengine.framework.dataspace.datablock.DataBlock method*), 34
get_header() (*decisionengine.framework.dataspace.datasource.DataSource method*), 36
get_header() (*decisionengine.framework.dataspace.datasources.postgresql.Postgresql method*), 29
get_header() (*decisionengine.framework.dataspace.dataspace.DataSpace method*), 38
get_last_generation_id() (*decisionengine.framework.dataspace.datasource.DataSource method*), 36
get_last_generation_id() (*decisionengine.framework.dataspace.datasources.postgresql.Postgresql method*), 29
get_last_generation_id() (*decisionengine.framework.dataspace.dataspace.DataSpace method*), 38
get_logger() (*decisionengine.framework.engine.DecisionEngine.DecisionEngine method*), 41
get_logger() (*in module decisionengine.framework.modules.de_logger*), 52
get_loglevel() (*decisionengine.framework.taskmanager.TaskManager.TaskManager method*), 55
get_metadata() (*decisionengine.framework.dataspace.datablock.DataBlock method*), 34
get_metadata() (*decisionengine.framework.dataspace.datasource.DataSource method*), 36
get_metadata() (*decisionengine.framework.dataspace.datasources.postgresql.Postgresql method*), 30
get_metadata() (*decisionengine.framework.dataspace.dataspace.DataSpace method*), 38
get_parameters() (*decisionengine.framework.modules.Module.Module method*), 51
get_produces() (*decisionengine.framework.engine.Workers.Worker method*), 43
get_produces() (*decisionengine.framework.taskmanager.TaskManager.TaskManager method*), 55
get_random_port() (*in module decisionengine.framework.util.sockets*), 64
get_schema() (*decisionengine.framework.dataspace.datasource.DataSource method*), 36
get_schema() (*decisionengine.framework.dataspace.datasources.postgresql.Postgresql method*), 30
get_state() (*decisionengine.framework.taskmanager.TaskManager.TaskManager method*), 55
get_state_name() (*decisionengine.framework.engine.Workers.Worker method*), 43
get_state_name() (*decisionengine.framework.taskmanager.TaskManager.TaskManager method*), 55
get_state_value() (*decisionengine.framework.taskmanager.TaskManager.TaskManager method*), 55
get_taskmanager() (*decisionengine.framework.dataspace.datablock.DataBlock method*), 34
get_taskmanager() (*decisionengine.framework.dataspace.datasource.DataSource method*), 37
get_taskmanager() (*decisionengine.framework.dataspace.datasources.postgresql.Postgresql method*), 30
get_taskmanager() (*decisionengine.framework.dataspace.dataspace.DataSpace method*), 38
get_taskmanagers() (*decisionengine.framework.dataspace.datasource.DataSource method*), 37

get_taskmanagers() (*decisionengine.framework.dataspace.datasources.postgresql.Postgresql method*), 30
get_taskmanagers() (*decisionengine.framework.dataspace.dataspace.DataSpace method*), 38
global_config_dir() (*in module decisionengine.framework.config.policies*), 26
global_config_file() (*in module decisionengine.framework.config.policies*), 26

H

handle_sighup() (*decisionengine.framework.engine.DecisionEngine.DecisionEngine method*), 41
has_value() (*decisionengine.framework.taskmanager.ProcessingState.ProcessingState method*), 54
Header (*class in decisionengine.framework.dataspace.datablock*), 34
header() (*in module decisionengine.framework.dataspace.datasources.tests.test_postgresql*), 27
header_table (*decisionengine.framework.dataspace.datasource.DataSource attribute*), 37
Help (*class in decisionengine.framework.tests.ModuleProgramOptions*), 58

I

IDLE (*decisionengine.framework.taskmanager.ProcessingState.State attribute*), 54
inactive() (*decisionengine.framework.taskmanager.ProcessingState.ProcessingState method*), 54
insert() (*decisionengine.framework.dataspace.datasource.DataSource method*), 37
insert() (*decisionengine.framework.dataspace.datasources.postgresql.Postgresql method*), 30
insert() (*decisionengine.framework.dataspace.dataspace.DataSpace method*), 38
InvalidMetadataError, 34
is_expired() (*decisionengine.framework.dataspace.datablock.DataBlock method*), 34
is_valid() (*decisionengine.framework.dataspace.datablock.Header method*), 34

K

keys() (*decisionengine.framework.dataspace.datablock.DataBlock method*), 34

L

load() (*in module decisionengine.framework.config.tests.test_config*), 23
load_all_channels() (*decisionengine.framework.config.ChannelConfigHandler.ChannelConfig method*), 25
load_channel() (*decisionengine.framework.config.ChannelConfigHandler.ChannelConfig method*), 25
lock (*decisionengine.framework.taskmanager.ProcessingState.ProcessingS property*), 54
logger (*decisionengine.framework.dataspace.tests.test_Reaper.TestReaper attribute*), 31
logic_engine_with_fact() (*in module decisionengine.framework.logicengine.tests.test_fail_on_error*), 46
LogicEngine (*class in decisionengine.framework.logicengine.LogicEngine*), 48
LogicEngine (*class in decisionengine.framework.modules.LogicEngine*), 50
LogicError, 47

M

main() (*in module decisionengine.framework.engine.de_client*), 44
main() (*in module decisionengine.framework.engine.de_query_tool*), 44
main() (*in module decisionengine.framework.engine.DecisionEngine*), 43
main() (*in module decisionengine.framework.util.reaper*), 64
main_wrapper() (*in module decisionengine.framework.modules.describe*), 53
make_db() (*in module decisionengine.framework.logicengine.tests.test_facts*), 45
mark_demented() (*decisionengine.framework.dataspace.dataspace.DataSpace method*), 38
mark_expired() (*decisionengine.framework.dataspace.datablock.DataBlock method*), 34
mark_expired() (*decisionengine.framework.dataspace.dataspace.DataSpace method*), 38
maybe_fail_on_error() (*in module decisionengine.framework.logicengine.BooleanExpression*), 47

Metadata (*class in decisionengine.framework.dataspace.datablock*), 34
metadata() (*in module decisionengine.framework.dataspace.datasources.tests.test_postgresql*), 27
metadata_table (*decisionengine.framework.dataspace.datasource.DataSource attribute*), 37
MIN_RETENTION_INTERVAL_DAYS (*decisionengine.framework.dataspace.maintain.Reaper attribute*), 39
MIN_SECONDS_BETWEEN_RUNS (*decisionengine.framework.dataspace.maintain.Reaper attribute*), 39
mock_data_block() (*in module decisionengine.framework.dataspace.datasources.tests.fixtures*), 27
MockSource (*class in decisionengine.framework.dataspace.tests.test_Reaper*), 31
module
decisionengine, 65
decisionengine.framework, 65
decisionengine.framework.about, 65
decisionengine.framework.config, 27
decisionengine.framework.config.ChannelConfigHandler, 25
decisionengine.framework.config.policies, 26
decisionengine.framework.config.tests, 25
decisionengine.framework.config.tests.test_config, 23
decisionengine.framework.config.tests.test_policies, 24
decisionengine.framework.config.tests.test_validconfig, 24
decisionengine.framework.config.ValidConfig, 25
decisionengine.framework.dataspace, 39
decisionengine.framework.dataspace.datablock, 33
decisionengine.framework.dataspace.datasource, 35
decisionengine.framework.dataspace.datasources, 31
decisionengine.framework.dataspace.datasources.postgresql, 28
decisionengine.framework.dataspace.datasources.tests, 28
decisionengine.framework.dataspace.datasources.tests.fixtures, 27
decisionengine.framework.dataspace.datasources.tests.test_postgresql, 27
decisionengine.framework.dataspace.dataspace, 38
decisionengine.framework.dataspace.maintain, 39
decisionengine.framework.dataspace.tests, 33
decisionengine.framework.dataspace.tests.test_databloc 32
decisionengine.framework.dataspace.tests.test_datasour 33
decisionengine.framework.dataspace.tests.test_dataspac 33
decisionengine.framework.dataspace.tests.test_Reaper, 31
decisionengine.framework.engine, 45
decisionengine.framework.engine.de_client, 44
decisionengine.framework.engine.de_query_tool, 44
decisionengine.framework.engine.DecisionEngine, 41
decisionengine.framework.engine.tests, 41
decisionengine.framework.engine.tests.fixtures, 39
decisionengine.framework.engine.tests.test_client_only 40
decisionengine.framework.engine.tests.test_query_tool_ 40
decisionengine.framework.engine.tests.test_startup, 41
decisionengine.framework.engine.Workers, 43
decisionengine.framework.logicengine, 49
decisionengine.framework.logicengine.BooleanExpression 47
decisionengine.framework.logicengine.FactLookup, 47
decisionengine.framework.logicengine.LogicEngine, 48
decisionengine.framework.logicengine.Rule, 48
decisionengine.framework.logicengine.RuleEngine, 49
decisionengine.framework.logicengine.tests, 47
decisionengine.framework.logicengine.tests.test_cascad 45
decisionengine.framework.logicengine.tests.test_constr 45
decisionengine.framework.logicengine.tests.test_duplic 45
decisionengine.framework.logicengine.tests.test_facts, 45
decisionengine.framework.logicengine.tests.test_fail_o

46
decisionengine.framework.logicengine.tests.test_pandas_fact, 46
decisionengine.framework.logicengine.tests.test_rule_with_negated_fact, 46
decisionengine.framework.logicengine.tests.test_simple_configuration, 46
decisionengine.framework.modules, 54
decisionengine.framework.modules.de_logger, 52
decisionengine.framework.modules.describe, 53
decisionengine.framework.modules.LogicEngine, 50
decisionengine.framework.modules.Module, 51
decisionengine.framework.modules.print_description, 53
decisionengine.framework.modules.Publisher, 51
decisionengine.framework.modules.Source, 51
decisionengine.framework.modules.SourceProxy, 52
decisionengine.framework.modules.tests, 50
decisionengine.framework.modules.tests.test_decision, 50
decisionengine.framework.modules.tests.test_LogicEngine, 49
decisionengine.framework.modules.tests.test_Module, 49
decisionengine.framework.modules.tests.test_module_decorators, 50
decisionengine.framework.modules.tests.test_Publisher, 49
decisionengine.framework.modules.tests.test_Source, 50
decisionengine.framework.modules.tests.test_Transform, 50
decisionengine.framework.modules.Transform, 52
decisionengine.framework.taskmanager, 57
decisionengine.framework.taskmanager.module_graph, 57
decisionengine.framework.taskmanager.ProcessingState, 54
decisionengine.framework.taskmanager.TaskManager, 55
decisionengine.framework.tests, 64
decisionengine.framework.tests.ABTransform, 57
decisionengine.framework.tests.BATransform, 57
decisionengine.framework.tests.FailingPublisher, 57
decisionengine.framework.tests.FailingSourceNOP, 58
decisionengine.framework.tests.FailingSourceProxy, 58
decisionengine.framework.tests.fixtures, 60
decisionengine.framework.tests.ModuleProgramOptions, 58
decisionengine.framework.tests.PublisherNOP, 59
decisionengine.framework.tests.PublisherWithMissingCon 59
decisionengine.framework.tests.SourceAlias, 59
decisionengine.framework.tests.SourceNOP, 59
decisionengine.framework.tests.SourceWithSampleConfigN 59
decisionengine.framework.tests.SupportsConfigPublisher 59
decisionengine.framework.tests.test_client_errors, 61
decisionengine.framework.tests.test_client_server, 61
decisionengine.framework.tests.test_defaults, 61
decisionengine.framework.tests.test_module_program_opt 61
decisionengine.framework.tests.test_query_tool_server, 61
decisionengine.framework.tests.test_reaper, 62
decisionengine.framework.tests.test_restart_channel, 62
decisionengine.framework.tests.test_sample_config, 62
decisionengine.framework.tests.test_source_proxy, 63
decisionengine.framework.tests.test_start_with_bad_cha 63
decisionengine.framework.tests.test_start_with_no_chan 64
decisionengine.framework.tests.TransformNOP, 60
decisionengine.framework.tests.TransformWithMissingPro 60
decisionengine.framework.tests.WorkingSourceProxy, 60
decisionengine.framework.util, 65
decisionengine.framework.util.fs, 64
decisionengine.framework.util.reaper, 64
decisionengine.framework.util.sockets, 64

decisionengine.framework.util.subclasses, 65
decisionengine.framework.version, 65
decisionengine.tests, 65
decisionengine.tests.test_framework_package, 65
Module (*class in decisionengine.framework.modules.Module*), 51
ModuleProgramOptions (*class in decisionengine.framework.modules.describe*), 53
mydata() (*in module decisionengine.framework.logicengine.tests.test_pandas_fact*), 46
myengine() (*in module decisionengine.framework.logicengine.tests.test_cascaded_rules*), 45
myengine() (*in module decisionengine.framework.logicengine.tests.test_pandas_fact*), 46
myengine() (*in module decisionengine.framework.logicengine.tests.test_rule_with_negated_fact*), 46
myengine() (*in module decisionengine.framework.logicengine.tests.test_simple_configuration*), 46

N

NotFound (*decisionengine.framework.engine.DecisionEngine.StopState attribute*), 42

O

OFFLINE (*decisionengine.framework.taskmanager.ProcessingState.State attribute*), 54

P

Parameter (*class in decisionengine.framework.modules.describe*), 53
Parameter (*class in decisionengine.framework.modules.Publisher*), 51
Parameter (*class in decisionengine.framework.modules.Source*), 51
Parameter (*class in decisionengine.framework.modules.Transform*), 52
parse_program_options() (*in module decisionengine.framework.engine.DecisionEngine*), 43
PG_PROG() (*in module decisionengine.framework.engine.tests.fixtures*), 40
PG_PROG() (*in module decisionengine.framework.tests.fixtures*), 60
post_create() (*decisionengine.framework.modules.Source.Source method*), 51
post_create() (*decisionengine.framework.modules.SourceProxy.SourceProxy method*), 52
Postgresql (*class in decisionengine.framework.dataspace.datasources.postgresql*), 28
print_channel_config() (*decisionengine.framework.config.ChannelConfigHandler.ChannelConfig method*), 25
print_consumes() (*in module decisionengine.framework.modules.print_description*), 53
print_produces() (*in module decisionengine.framework.modules.print_description*), 53
print_supported_config() (*in module decisionengine.framework.modules.print_description*), 53
probably_running() (*decisionengine.framework.taskmanager.ProcessingState.ProcessingState method*), 54
process_args() (*decisionengine.framework.modules.describe.ModuleProgramOptions method*), 53
ProcessingState (*class in decisionengine.framework.taskmanager.ProcessingState*), 54
produces() (*decisionengine.framework.logicengine.LogicEngine.LogicEn method*), 48
produces() (*in module decisionengine.framework.modules.Module*), 51
produces() (*in module decisionengine.framework.modules.Source*), 51
produces() (*in module decisionengine.framework.modules.Transform*), 52
ProductRetriever (*class in decisionengine.framework.dataspace.datablock*), 35
publish() (*decisionengine.framework.modules.Publisher.Publisher method*), 51
publish() (*decisionengine.framework.tests.FailingPublisher.FailingPublis method*), 57
publish() (*decisionengine.framework.tests.PublisherNOP.PublisherNOP method*), 59
Publisher (*class in decisionengine.framework.modules.Publisher*), 51
PublisherNOP (*class in decisionengine.framework.tests.PublisherNOP*), 59

PublisherWithMissingConsumes (*class in decisionengine.framework.tests.PublisherWithMissingConsumes*), 59
put() (*decisionengine.framework.dataspace.datablock.DataBlock method*), 34
pytestmark (*decisionengine.framework.dataspace.tests.test_datablock.TestDatablock attribute*), 32

R

reap() (*decisionengine.framework.dataspace.maintain.Reaper method*), 39
Reaper (*class in decisionengine.framework.dataspace.maintain*), 39
reaper_start() (*decisionengine.framework.engine.DecisionEngine.DecisionEngine method*), 41
reaper_status() (*decisionengine.framework.engine.DecisionEngine.DecisionEngine method*), 41
reaper_stop() (*decisionengine.framework.engine.DecisionEngine.DecisionEngine method*), 41
RequestHandler (*class in decisionengine.framework.engine.DecisionEngine*), 42
required_keys (*decisionengine.framework.dataspace.datablock.Header attribute*), 34
required_keys (*decisionengine.framework.dataspace.datablock.Metadata attribute*), 35
retention_interval (*decisionengine.framework.dataspace.maintain.Reaper property*), 39
rm_channel() (*decisionengine.framework.engine.DecisionEngine.DecisionEngine method*), 41
rpc_block_while() (*decisionengine.framework.engine.DecisionEngine.DecisionEngine method*), 41
rpc_get_channel_log_level() (*decisionengine.framework.engine.DecisionEngine.DecisionEngine method*), 42
rpc_get_log_level() (*decisionengine.framework.engine.DecisionEngine.DecisionEngine method*), 42
rpc_kill_channel() (*decisionengine.framework.engine.DecisionEngine.DecisionEngine method*), 42
rpc_paths (*decisionengine.framework.engine.DecisionEngine.RequestHandler attribute*), 42
rpc_print_product() (*decisionengine.framework.engine.DecisionEngine.DecisionEngine method*), 42
rpc_print_products() (*decisionengine.framework.engine.DecisionEngine.DecisionEngine method*), 42
rpc_query_tool() (*decisionengine.framework.engine.DecisionEngine.DecisionEngine method*), 42
rpc_reaper_start() (*decisionengine.framework.engine.DecisionEngine.DecisionEngine method*), 42
rpc_reaper_status() (*decisionengine.framework.engine.DecisionEngine.DecisionEngine method*), 42
rpc_reaper_stop() (*decisionengine.framework.engine.DecisionEngine.DecisionEngine method*), 42
rpc_rm_channel() (*decisionengine.framework.engine.DecisionEngine.DecisionEngine method*), 42
rpc_set_channel_log_level() (*decisionengine.framework.engine.DecisionEngine.DecisionEngine method*), 42
rpc_show_config() (*decisionengine.framework.engine.DecisionEngine.DecisionEngine method*), 42
rpc_show_de_config() (*decisionengine.framework.engine.DecisionEngine.DecisionEngine method*), 42
rpc_start_channel() (*decisionengine.framework.engine.DecisionEngine.DecisionEngine method*), 42
rpc_start_channels() (*decisionengine.framework.engine.DecisionEngine.DecisionEngine method*), 42
rpc_status() (*decisionengine.framework.engine.DecisionEngine.DecisionEngine method*), 42
rpc_stop() (*decisionengine.framework.engine.DecisionEngine.DecisionE method*), 42
rpc_stop_channel() (*decisionengine.framework.engine.DecisionEngine.DecisionEngine method*), 42
rpc_stop_channels() (*decisionengine.framework.engine.DecisionEngine.DecisionEngine method*), 42
Rule (*class in decisionengine.framework.logicengine.Rule*), 48
rule_for() (*decisionengine.framework.logicengine.FactLookup.FactLook method*), 47
RuleEngine (*class in decisionengine.framework.logicengine.RuleEngine*), 49
run() (*decisionengine.framework.engine.Workers.Worker method*), 43

run() (*decisionengine.framework.taskmanager.TaskManager.TaskManager method*), 55
run_logic_engine() (*decisionengine.framework.taskmanager.TaskManager.TaskManager method*), 55
run_publishers() (*decisionengine.framework.taskmanager.TaskManager.TaskManager method*), 55
run_source() (*decisionengine.framework.taskmanager.TaskManager.TaskManager method*), 55
run_transform() (*decisionengine.framework.taskmanager.TaskManager.TaskManager method*), 56
run_transforms() (*decisionengine.framework.taskmanager.TaskManager.TaskManager method*), 56

S

seconds_between_runs (*decisionengine.framework.dataspace.maintain.Reaper property*), 39
set() (*decisionengine.framework.taskmanager.ProcessingState.ProcessingState method*), 54
set_data_block() (*decisionengine.framework.modules.Module.Module method*), 51
set_logging() (*in module decisionengine.framework.modules.de_logger*), 52
set_loglevel_value() (*decisionengine.framework.taskmanager.TaskManager.TaskManager method*), 56
set_state() (*decisionengine.framework.dataspace.datablock.Metadata method*), 35
set_stream_logging() (*in module decisionengine.framework.modules.de_logger*), 53
setUp() (*decisionengine.framework.dataspace.tests.test_datablock.TestDataBlock method*), 32
setUp() (*decisionengine.framework.dataspace.tests.test_Reaper.TestReaper method*), 31
setUp() (*decisionengine.framework.modules.tests.test_de_logger.TestLogger method*), 50
should_stop() (*decisionengine.framework.taskmanager.ProcessingState.ProcessingState method*), 54
SHUTDOWN (*decisionengine.framework.taskmanager.ProcessingState.State attribute*), 55
shutdown() (*decisionengine.framework.modules.Publisher.Publisher method*), 51
SHUTTINGDOWN (*decisionengine.framework.taskmanager.ProcessingState.State attribute*), 55
Singleton (*class in decisionengine.framework.dataspace.dataspace*), 38
sorted_rules() (*decisionengine.framework.logicengine.FactLookup.FactLookup method*), 47
Source (*class in decisionengine.framework.modules.Source*), 51
SourceNOP (*class in decisionengine.framework.tests.SourceNOP*), 59
SourceProxy (*class in decisionengine.framework.modules.SourceProxy*), 52
SourceWithMissingProduces (*class in decisionengine.framework.tests.FailingSourceNOP*), 58
SourceWithSampleConfigNOP (*class in decisionengine.framework.tests.SourceWithSampleConfigNOP*), 59
spec_if_main() (*in module decisionengine.framework.modules.print_description*), 53
start() (*decisionengine.framework.dataspace.maintain.Reaper method*), 39
start_channel() (*decisionengine.framework.engine.DecisionEngine.DecisionEngine method*), 42
start_channels() (*decisionengine.framework.engine.DecisionEngine.DecisionEngine method*), 42
start_sources() (*decisionengine.framework.taskmanager.TaskManager.TaskManager method*), 56
State (*class in decisionengine.framework.taskmanager.ProcessingState*), 54
STEADY (*decisionengine.framework.taskmanager.ProcessingState.State attribute*), 55
stop() (*decisionengine.framework.dataspace.maintain.Reaper method*), 39
stop_channels() (*decisionengine.framework.engine.DecisionEngine.DecisionEngine method*), 42
stop_worker() (*decisionengine.framework.engine.DecisionEngine.DecisionEngine method*), 42
StopState (*class in decisionengine.framework.engine.DecisionEngine*), 42
store_taskmanager() (*decisionengine.framework.dataspace.datablock.DataBlock method*), 34
store_taskmanager() (*decisio-*

nengine.framework.dataspace.datasource.DataSource *method*), 37
store_taskmanager() (*decisionengine.framework.dataspace.datasources.postgresql.Postgresql* *method*), 30
store_taskmanager() (*decisionengine.framework.dataspace.dataspace.DataSpace* *method*), 38
supports_config() (*in module decisionengine.framework.modules.describe*), 53
supports_config() (*in module decisionengine.framework.modules.Publisher*), 51
supports_config() (*in module decisionengine.framework.modules.Source*), 51
supports_config() (*in module decisionengine.framework.modules.Transform*), 52
SupportsConfig (*class in decisionengine.framework.tests.SupportsConfigPublisher*), 59

T

tables (*decisionengine.framework.dataspace.datasources.postgresql.Postgresql* *attribute*), 30
take_offline() (*decisionengine.framework.taskmanager.TaskManager.TaskManager* *method*), 56
TaskManager (*class in decisionengine.framework.taskmanager.TaskManager*), 55
taskmanager() (*in module decisionengine.framework.dataspace.datasources.tests.test_postgresql*), 27
taskmanager_table (*decisionengine.framework.dataspace.datasource.DataSource* *attribute*), 37
tearDown() (*decisionengine.framework.dataspace.tests.test_datablockTestDataframework* *method*), 32
tearDown() (*decisionengine.framework.dataspace.tests.test_RetryChannelLoader* *method*), 31
Terminated (*decisionengine.framework.engine.DecisionEngine.StopState* *attribute*), 42
test() (*decisionengine.framework.tests.ModuleProgramOptions.AcquireWithConfig* *method*), 58
test() (*decisionengine.framework.tests.ModuleProgramOptions.AcquireWithSampleConfig* *method*), 58
test() (*decisionengine.framework.tests.ModuleProgramOptions.ConfigTemplate* *method*), 58
test() (*decisionengine.framework.tests.ModuleProgramOptions.Describe* *method*), 58
test() (*decisionengine.framework.tests.ModuleProgramOptions.DescribeAlias* *method*), 58
test() (*decisionengine.framework.tests.ModuleProgramOptions.Help* *method*), 58
test_acquire_for_sources() (*in module decisionengine.framework.tests.test_module_program_options*), 61
test_by_nonsense_is_err() (*decisionengine.framework.modules.tests.test_de_logger.TestLogger* *method*), 50
test_by_size() (*decisionengine.framework.modules.tests.test_de_logger.TestLogger* *method*), 50
test_by_time() (*decisionengine.framework.modules.tests.test_de_logger.TestLogger* *method*), 50
test_can_import() (*in module decisionengine.tests.test_framework_package*), 65
test_change_port() (*in module decisionengine.framework.engine.tests.test_startup*), 41
test_channel_config_dir() (*in module decisionengine.framework.config.tests.test_policies*), 24
test_channel_empty_config() (*in module decisionengine.framework.config.tests.test_config*), 23
test_channel_empty_dictionary() (*in module decisionengine.framework.config.tests.test_config*), 23
test_channel_invalid_modules_list() (*in module decisionengine.framework.config.tests.test_config*), 23
test_channel_invalid_modules_no_keys() (*in module decisionengine.framework.config.tests.test_config*), 23
test_channel_invalid_modules_string() (*in module decisionengine.framework.config.tests.test_config*), 23
test_channel_loading() (*in module decisionengine.framework.config.tests.test_config*), 23
test_channel_module_missing_all() (*in module decisionengine.framework.config.tests.test_config*), 24
test_channel_module_missing_module() (*in module decisionengine.framework.config.tests.test_config*), 24
test_channel_module_missing_parameters() (*in module decisionengine.framework.config.tests.test_config*), 24

test_channel_names() (*in module decisionengine.framework.config.tests.test_config*), 24
test_channel_no_config_files() (*in module decisionengine.framework.config.tests.test_config*), 24
test_channel_no_modules() (*in module decisionengine.framework.config.tests.test_config*), 24
test_client_can_double_set_de_server_channel_log_level() (*in module decisionengine.framework.tests.test_sample_config*), 62
test_client_can_get_de_server_channel_config() (*in module decisionengine.framework.tests.test_sample_config*), 62
test_client_can_get_de_server_channel_log_level() (*in module decisionengine.framework.tests.test_sample_config*), 62
test_client_can_get_de_server_reaper_start_delay() (*in module decisionengine.framework.tests.test_reaper*), 62
test_client_can_get_de_server_reaper_status() (*in module decisionengine.framework.tests.test_reaper*), 62
test_client_can_get_de_server_reaper_stop() (*in module decisionengine.framework.tests.test_reaper*), 62
test_client_can_get_de_server_show_channel_logger_level() (*in module decisionengine.framework.tests.test_defaults*), 61
test_client_can_get_de_server_show_config() (*in module decisionengine.framework.tests.test_sample_config*), 62
test_client_can_get_de_server_show_logger_level() (*in module decisionengine.framework.tests.test_sample_config*), 62
test_client_can_get_de_server_status() (*in module decisionengine.framework.tests.test_sample_config*), 62
test_client_can_get_products() (*in module decisionengine.framework.tests.test_sample_config*), 62
test_client_can_get_products_no_channels() (*in module decisionengine.framework.tests.test_sample_config*), 62
test_client_can_get_products_no_channels() (*in module decisionengine.framework.tests.test_start_with_bad_channels*), 63
test_client_can_kill_one_channel() (*in module decisionengine.framework.tests.test_sample_config*), 62
test_client_can_kill_one_channel_force() (*in module decisionengine.framework.tests.test_sample_config*), 62
test_client_can_kill_one_channel_timeout() (*in module decisionengine.framework.tests.test_sample_config*), 62
test_client_can_set_de_server_channel_log_level() (*in module decisionengine.framework.tests.test_sample_config*), 62
test_client_can_start_all_channel() (*in module decisionengine.framework.tests.test_sample_config*), 62
test_client_can_start_one_channel() (*in module decisionengine.framework.tests.test_sample_config*), 63
test_client_can_stop_channels() (*in module decisionengine.framework.tests.test_sample_config*), 63
test_client_can_stop_one_channel() (*in module decisionengine.framework.tests.test_sample_config*), 63
test_client_can_stop_server() (*in module decisionengine.framework.tests.test_sample_config*), 63
test_client_cannot_double_start() (*in module decisionengine.framework.tests.test_sample_config*), 63
test_client_cannot_wait_on_bad_state() (*in module decisionengine.framework.tests.test_client_errors*), 61
test_client_de_config_is_json() (*in module decisionengine.framework.tests.test_defaults*), 61
test_client_err_returned_as_rc() (*in module decisionengine.framework.engine.tests.test_client_only*), 40
test_client_err_returned_verbose_as_rc() (*in module decisio-*

nengine.framework.engine.tests.test_client_only), 40
`test_client_get_channel_log_fails_cleanly()` (*in module decisionengine.framework.tests.test_sample_config*), 63
`test_client_get_non_real_channel()` (*in module decisionengine.framework.tests.test_sample_config*), 63
`test_client_help()` (*in module decisionengine.framework.engine.tests.test_client_only*), 40
`test_client_print_product()` (*in module decisionengine.framework.tests.test_client_server*), 61
`test_client_set_channel_log_fails_cleanly()` (*in module decisionengine.framework.tests.test_sample_config*), 63
`test_client_start_non_real_channel()` (*in module decisionengine.framework.tests.test_sample_config*), 63
`test_client_status_msg_to_stdout()` (*in module decisionengine.framework.tests.test_client_server*), 61
`test_client_stop_non_real_channel()` (*in module decisionengine.framework.tests.test_sample_config*), 63
`test_client_with_no_command_says_use_help()` (*in module decisionengine.framework.engine.tests.test_client_only*), 40
`test_client_with_no_server()` (*in module decisionengine.framework.engine.tests.test_client_only*), 40
`test_client_with_no_server_verbose()` (*in module decisionengine.framework.engine.tests.test_client_only*), 40
`test_compound_fact_with_spaces()` (*in module decisionengine.framework.logicengine.tests.test_facts*), 45
`test_compress()` (*decisionengine.framework.dataspace.tests.test_datablock.TestDatablock method*), 32
`test_conditional_fact()` (*in module decisionengine.framework.logicengine.tests.test_fail_on_error*), 46
`test_config_templates()` (*in module decisionengine.framework.tests.test_module_program_options*), 61
`test_configuration_with_fact_using_function()` (*in module decisionengine.framework.logicengine.tests.test_construction*), 45
`test_configuration_with_numy_facts()` (*in module decisionengine.framework.logicengine.tests.test_construction*), 45
`test_create_tables()` (*in module decisionengine.framework.dataspace.datasources.tests.test_postgresql*), 27
`test_DataBlock_constructor()` (*decisionengine.framework.dataspace.tests.test_datablock.TestDatablock method*), 32
`test_DataBlock_duplicate()` (*decisionengine.framework.dataspace.tests.test_datablock.TestDatablock method*), 32
`test_DataBlock_get_dataproducts()` (*decisionengine.framework.dataspace.tests.test_datablock.TestDatablock method*), 32
`test_DataBlock_get_header()` (*decisionengine.framework.dataspace.tests.test_datablock.TestDatablock method*), 32
`test_DataBlock_get_metadata()` (*decisionengine.framework.dataspace.tests.test_datablock.TestDatablock method*), 32
`test_DataBlock_get_taskmanager()` (*decisionengine.framework.dataspace.tests.test_datablock.TestDatablock method*), 32
`test_DataBlock_get_taskmanagers()` (*decisionengine.framework.dataspace.tests.test_datablock.TestDatablock method*), 32
`test_DataBlock_is_expired()` (*decisionengine.framework.dataspace.tests.test_datablock.TestDatablock method*), 32
`test_DataBlock_is_expired_with_key()` (*decisionengine.framework.dataspace.tests.test_datablock.TestDatablock method*), 32
`test_DataBlock_key_management()` (*decisionengine.framework.dataspace.tests.test_datablock.TestDatablock method*), 32
`test_DataBlock_mark_expired()` (*decisionengine.framework.dataspace.tests.test_datablock.TestDatablock method*), 32
`test_DataBlock_to_str()` (*decisionengine.framework.dataspace.tests.test_datablock.TestDatablock method*), 32
`test_dataspace_config_finds_bad()` (*in module decisionengine.framework.dataspace.tests.test_dataspace*), 33
`test_default_config()` (*in module decisionengine.framework.engine.tests.test_startup*),

41
test_default_construction() (*in module decisionengine.framework.logicengine.tests.test_construction*), 45
test_delete_data_older_than_arg() (*in module decisionengine.framework.dataspace.datasources.tests.test_postgresql*), 27
test_descriptions() (*in module decisionengine.framework.tests.test_module_program_options*), 61
test_duplicate_fact_names() (*in module decisionengine.framework.logicengine.tests.test_duplicate_fact_names*), 45
test_empty_config() (*in module decisionengine.framework.config.tests.test_config*), 24
test_empty_config() (*in module decisionengine.framework.config.tests.test_validconfig*), 24
test_empty_dict() (*in module decisionengine.framework.config.tests.test_config*), 24
test_empty_dict_with_leading_comment() (*in module decisionengine.framework.config.tests.test_config*), 24
test_exclusive_options() (*in module decisionengine.framework.engine.tests.test_client_only*), 40
test_fact_using_numpy_array() (*in module decisionengine.framework.logicengine.tests.test_facts*), 45
test_fact_using_numpy_function() (*in module decisionengine.framework.logicengine.tests.test_facts*), 45
test_fact_with_fail_on_error() (*in module decisionengine.framework.logicengine.tests.test_facts*), 45
test_fact_with_misspecified_attribute() (*in module decisionengine.framework.logicengine.tests.test_fail_on_error*), 46
test_fact_with_nested_names() (*in module decisionengine.framework.logicengine.tests.test_facts*), 45
test_fail_bad_config() (*decisionengine.framework.dataspace.tests.test_Reaper.TestReaper method*), 31
test_fail_missing_config() (*decisionengine.framework.dataspace.tests.test_Reaper.TestReaper method*), 31
test_fail_missing_config_key() (*decisionengine.framework.dataspace.tests.test_Reaper.TestReaper method*), 31
test_fail_on_error() (*in module decisionengine.framework.logicengine.tests.test_fail_on_error*), 46
test_fail_small_retain() (*decisionengine.framework.dataspace.tests.test_Reaper.TestReaper method*), 31
test_fail_small_run_interval() (*decisionengine.framework.dataspace.tests.test_Reaper.TestReaper method*), 31
test_fail_start_two_reapers() (*decisionengine.framework.dataspace.tests.test_Reaper.TestReaper method*), 31
test_fail_wrong_config_key() (*decisionengine.framework.dataspace.tests.test_Reaper.TestReaper method*), 31
test_false_fact_with_spaces() (*in module decisionengine.framework.logicengine.tests.test_fail_on_error*), 46
test_false_literal_fact() (*in module decisionengine.framework.logicengine.tests.test_fail_on_error*), 46
test_generate_insert_query() (*in module decisionengine.framework.dataspace.datasources.tests.test_postgresql*), 27
test_get_dataproducts() (*in module decisionengine.framework.dataspace.datasources.tests.test_postgresql*), 27
test_get_last_generation_id() (*in module decisionengine.framework.dataspace.datasources.tests.test_postgresql*), 28
test_get_taskmanager() (*in module decisionengine.framework.dataspace.datasources.tests.test_postgresql*), 28
test_global_channel_log_level_in_config() (*in module decisionengine.framework.tests.test_defaults*), 61
test_global_config_dir() (*in module decisionengine.framework.config.tests.test_policies*), 24
test_global_config_file() (*in module decisionengine.framework.config.tests.test_policies*), 24
test_has_methods_we_expect() (*in module decisionengine.framework.dataspace.tests.test_datasource*), 33
test_Header_constructor() (*decisionengine.framework.dataspace.tests.test_datablock.TestDatablock method*), 32
test_Header_is_valid() (*decisionengine.framework.dataspace.tests.test_datablock.TestDatablock method*), 32

test_help() (*in module decisionengine.framework.tests.test_module_program_options*), 61
test_index_error() (*in module decisionengine.framework.logicengine.tests.test_fail_on_error*), 46
test_insert() (*in module decisionengine.framework.dataspace.datasources.tests.test_postgresql*), 28
test_invalid_config() (*in module decisionengine.framework.config.tests.test_validconfig*), 24
test_just_stop_no_error() (*decisionengine.framework.dataspace.tests.test_Reaper.TestReaper method*), 31
test_logicengine_structure() (*in module decisionengine.framework.modules.tests.test_LogicEngine*), 49
test_loop_of_start_stop_in_clumps() (*decisionengine.framework.dataspace.tests.test_Reaper.TestReaper method*), 31
test_Metadata_constructor() (*decisionengine.framework.dataspace.tests.test_datablock.TestDatablock method*), 32
test_Metadata_set_state() (*decisionengine.framework.dataspace.tests.test_datablock.TestDatablock method*), 32
test_minimal_jsonnet_right_extension() (*in module decisionengine.framework.config.tests.test_config*), 24
test_minimal_jsonnet_wrong_extension() (*in module decisionengine.framework.config.tests.test_config*), 24
test_minimal_python() (*in module decisionengine.framework.config.tests.test_config*), 24
test_misspecified_fact() (*in module decisionengine.framework.logicengine.tests.test_fail_on_error*), 46
test_module_alias() (*in module decisionengine.framework.tests.test_module_program_options*), 61
test_module_structure() (*in module decisionengine.framework.modules.tests.test_Module*), 49
test_multiple_consumes_declarations() (*in module decisionengine.framework.modules.tests.test_module_decorators*), 50
test_multiple_produces_declarations() (*in module decisionengine.framework.modules.tests.test_module_decorators*), 50
test_no_such_file() (*in module decisionengine.framework.config.tests.test_validconfig*), 24
test_publisher_structure() (*in module decisionengine.framework.modules.tests.test_Publisher*), 49
test_query_tool_csv() (*in module decisionengine.framework.tests.test_query_tool_server*), 61
test_query_tool_default() (*in module decisionengine.framework.tests.test_query_tool_server*), 61
test_query_tool_help() (*in module decisionengine.framework.engine.tests.test_query_tool_only*), 40
test_query_tool_invalid_product() (*in module decisionengine.framework.tests.test_query_tool_server*), 61
test_query_tool_json() (*in module decisionengine.framework.tests.test_query_tool_server*), 61
test_query_tool_since() (*in module decisionengine.framework.tests.test_query_tool_server*), 61
test_query_tool_with_no_server() (*in module decisionengine.framework.engine.tests.test_query_tool_only*), 40
test_query_tool_with_no_server_verbose() (*in module decisionengine.framework.engine.tests.test_query_tool_only*), 40
test_reap_default_state() (*decisionengine.framework.dataspace.tests.test_Reaper.TestReaper method*), 31
test_reaper_can_reap() (*decisionengine.framework.dataspace.tests.test_Reaper.TestReaper method*), 31
test_restart_channel() (*in module decisionengine.framework.tests.test_restart_channel*), 62
test_rule_that_does_not_fire() (*in module decisionengine.framework.logicengine.tests.test_cascaded_rules*), 45
test_rule_that_does_not_fire() (*in module decisionengine.framework.logicengine.tests.test_pandas_fact*), 46
test_rule_that_does_not_fire() (*in module decisionengine.framework.logicengine.tests.test_rule_with_negated_fact*

46
test_rule_that_does_not_fire() (*in module decisionengine.framework.logicengine.tests.test_simple_configuration*), 46
test_rule_that_fires() (*in module decisionengine.framework.logicengine.tests.test_cascaded_rules*), 45
test_rule_that_fires() (*in module decisionengine.framework.logicengine.tests.test_pandas_fact*), 46
test_rule_that_fires() (*in module decisionengine.framework.logicengine.tests.test_rule_with_negated_fact*), 46
test_rule_that_fires() (*in module decisionengine.framework.logicengine.tests.test_simple_configuration*), 46
test_simple_fact() (*in module decisionengine.framework.logicengine.tests.test_facts*), 45
test_source_fail_can_be_fixed() (*decisionengine.framework.dataspace.tests.test_Reaper.TestReaper method*), 31
test_source_structure() (*in module decisionengine.framework.modules.tests.test_Source*), 50
test_start_delay() (*decisionengine.framework.dataspace.tests.test_Reaper.TestReaper method*), 32
test_start_from_nothing() (*in module decisionengine.framework.tests.test_start_with_no_channels*), 64
test_start_stop() (*decisionengine.framework.dataspace.tests.test_Reaper.TestReaper method*), 32
test_start_stop_stop() (*decisionengine.framework.dataspace.tests.test_Reaper.TestReaper method*), 32
test_state_can_be_active() (*decisionengine.framework.dataspace.tests.test_Reaper.TestReaper method*), 32
test_state_sets_timer_and_uses_it() (*decisionengine.framework.dataspace.tests.test_Reaper.TestReaper method*), 32
test_stop_failing_source_proxy() (*in module decisionengine.framework.tests.test_source_proxy*), 63
test_store_taskmanager() (*in module decisionengine.framework.dataspace.datasources.tests.test_postgresql*), 28
test_supports_config() (*in module decisionengine.framework.modules.tests.test_module_decorators*), 50
test_syntax_error() (*in module decisionengine.framework.logicengine.tests.test_facts*), 45
test_syntax_error_in_config_names_bad_file() (*in module decisionengine.framework.config.tests.test_config*), 24
test_transform_structure() (*in module decisionengine.framework.modules.tests.test_Transform*), 50
test_trivial_configuration() (*in module decisionengine.framework.logicengine.tests.test_construction*), 45
test_true_fact() (*in module decisionengine.framework.logicengine.tests.test_fail_on_error*), 46
test_true_literal_fact() (*in module decisionengine.framework.logicengine.tests.test_fail_on_error*), 46
test_valid_dir() (*in module decisionengine.framework.config.tests.test_policies*), 24
test_working_source_proxy() (*in module decisionengine.framework.tests.test_source_proxy*), 63
test_wrong_configuration() (*in module decisionengine.framework.logicengine.tests.test_construction*), 45
test_wrong_product_names() (*in module decisionengine.framework.modules.tests.test_module_decorators*), 50
test_wrong_product_types() (*in module decisionengine.framework.modules.tests.test_module_decorators*), 50
test_wrong_type() (*in module decisionengine.framework.config.tests.test_config*), 24
test_wrong_type_config() (*in module decisionengine.framework.config.tests.test_validconfig*), 24
test_zdumps() (*decisionengine.framework.dataspace.tests.test_datablock.TestDatablock method*), 32
test_zloads() (*decisionengine.framework.dataspace.tests.test_datablock.TestDatablock method*), 32
TestDatablock (*class in decisionengine.framework.dataspace.tests.test_datablock*), 32
TestLogger (*class in decisionengine.framework.modules.tests.test_de_logger*), 50
TestReaper (*class in decisionengine.framework.dataspace.tests.test_Reaper*), 31

Transform (*class in decisionengine.framework.modules.Transform*), 52
transform() (*decisionengine.framework.modules.Transform.Transform method*), 52
transform() (*decisionengine.framework.tests.TransformNOP.TransformNOP method*), 60
transform() (*decisionengine.framework.tests.TransformWithMissingProducesConsumes.TransformWithMissingProducesConsumes method*), 60
TransformNOP (*class in decisionengine.framework.tests.TransformNOP*), 60
TransformWithMissingProducesConsumes (*class in decisionengine.framework.tests.TransformWithMissingProducesConsumes*), 60

U

unguarded_access() (*decisionengine.framework.engine.Workers.Workers method*), 44
update() (*decisionengine.framework.dataspace.datasource.DataSource method*), 37
update() (*decisionengine.framework.dataspace.datasources.postgresql.Postgresql method*), 30
update() (*decisionengine.framework.dataspace.dataspace.DataSpace method*), 38

V

valid_dir() (*in module decisionengine.framework.config.policies*), 26
valid_states (*decisionengine.framework.dataspace.datablock.Metadata attribute*), 35
ValidConfig (*class in decisionengine.framework.config.ValidConfig*), 25
verify_products() (*in module decisionengine.framework.modules.Module*), 51

W

wait_for_all() (*decisionengine.framework.taskmanager.TaskManager.TaskManager method*), 56
wait_for_any() (*decisionengine.framework.taskmanager.TaskManager.TaskManager method*), 56
wait_until() (*decisionengine.framework.engine.Workers.Worker method*), 43
wait_until() (*decisionengine.framework.taskmanager.ProcessingState.ProcessingState method*), 54
wait_while() (*decisionengine.framework.engine.Workers.Worker method*), 43
wait_while() (*decisionengine.framework.taskmanager.ProcessingState.ProcessingState method*), 54
Worker (*class in decisionengine.framework.engine.Workers*), 43
Worker (*class in decisionengine.framework.taskmanager.TaskManager*), 56
Workers (*class in decisionengine.framework.engine.Workers*), 43
Workers.Access (*class in decisionengine.framework.engine.Workers*), 44
WorkingSourceProxy (*class in decisionengine.framework.tests.WorkingSourceProxy*), 60

Z

zdumps() (*in module decisionengine.framework.dataspace.datablock*), 35
zloads() (*in module decisionengine.framework.dataspace.datablock*), 35